简介

2025年4月21日 0:34

综合 75道选择 45/75 150min

案例 3道 1个选2 45/75 90min

论文 1个选1 45/75 120min

第一篇：数学、计算机系统、网络、数据库、企业信息化、软件工程、项目管理、信息安全

第二篇：系统规划 设计、软件需求工程、软件架构设计、系统设计、软件测试 测评、系统运行维护

web 嵌入式 移动应用 大数据 微服务 信息物理系统

视频课程目录	第二版教材对应章节及知识点	考分
1.计算机系统	3.1-3.5：概述、存储器、输入输出、指令系统、多处理机系统	6
2.操作系统	3.6：进程管理、存储管理、设备管理、文件管理、作业管理	6
3.数据库技术	5：DBMS、关系数据库、并发控制、设计建模、分布式数据库、数据仓库、数据挖掘、非关系数据库。	6
4.计算机网络与分布式系统	4：网络基础、网络架构与协议、局域网和广域网、网络工程、分布式系统、构件与中间件、Web服务、云计算。	6
5.信息安全	9：安全体系、保密、网络安全、访问控制、容灾与业务持续、安全管理、可靠性、冗余技术、容错技术	6
6.企业信息化	6：概述、信息资源管理、信息化规划、企业信息系统、电子商务、电子政务、业务流程重组、企业应用集成	6
7.软件工程	7：生命周期、开发方法和模型、开发环境和工具、软件过程管理、软件重用和再工程 软件产品线、统一建模语言、软件形式化方法	3
8.项目管理	8：概述、范围管理、进度管理、成本管理、配置管理、质量管理、人力资源管理、风险管理、信息（文档）管理	2
9.系统规划与分析	10：概述、项目提出与选择、系统分析概述、问题分析、业务流程分析、数据与数据流程分析、系统可行性分析、成本效益分析、系统方案建议	1
10.软件需求工程	11：概述、需求获取、分析、结构化分析、面向对象分析、需求定义、验证、管理	3
11.软件架构设计	12：概述、架构建模、架构风格、架构标准、架构实现、质量属性	3

视频课程目录	第二版教材对应章节及知识点	考分
12.系统设计	13：概述、处理流程设计、结构化设计、面向对象设计、设计模式、输入/输出原型设计、人机交互设计	3
13.软件实现与测试	14：软件实现概述、测试概述、测试方法、测试类型、测试组织、部署	4
14.系统运行与维护	15：运维技术指标、运行管理、故障管理、软件维护、系统评价、遗留系统处置、新旧系统转换、现有系统演进。	1
15.Web应用系统分析与设计	16：Web应用系统简介、架构设计、开发框架、开发、测试	在案例分析选择题中考查，可能考几分
16.嵌入式系统分析与设计	17：嵌入式系统概述、数据库、操作系统、开发、验证	
17.移动应用系统分析与设计	18：移动应用平台、开发环境、架构、开发、无代码开发	
18.大数据处理系统分析与设计	19：大数据处理系统概述、架构、开发、测试	
19.微服务系统分析与设计	20：微服务系统概述、架构、开发、测试	
20.信息物理系统分析与设计	21：信息物理系统概述、架构、技术框架、开发技术、控制系统与网络通信、应用分析与设计	
21.数据与经济管理	1+2：绪论、数学统计基础、图论应用、预测与决策、数学建模、工程伦理	3
补充：知识产权	虽然新版删除，但考的概率很大，必须学	3
补充：系统性能	新版删除，有考的可能，建议学	1
专业英语	无视频讲解，有总结英语单词和真题，自己练习记忆	5

第二版考纲：web应用系统分析与设计、嵌入式系统分析与设计、移动应用系统分析与设计、大数据处理系统分析与设计、微服务系统分析与设计、信息物理系统分析与设计。

新考纲其实就是第二版教材第三篇的内容，然而传统的七大类案例考点还是大概率会考，双管齐下。

案例分析真题	考点分布
1.系统规划	步骤，机会选择，问题分析，可行性分析，成本效益分析，净现值计算
2.系统分析和需求工程	数据流图、UML图（用例图、状态图、类图、活动图、时序图）、需求获取、需求分析、需求定义（需求规格说明书内容）、需求验证、需求管理和跟踪、需求变更（流程）、逆向工程、软件重构。
3.系统运行和维护	遗留系统，系统转换，数据迁移，软件维护，冗余技术，容错技术，集群技术，负载均衡。
4.项目管理	PERT图，甘特图，关键路径计算
5.数据库系统	数据库设计、E-R模型、规范化、事务并发控制和锁机制、反规划范、分布式数据库、数据库性能优化、数据仓库、数据挖掘、NoSQL。
6.嵌入式	嵌入式微处理器、嵌入式软件操作系统、嵌入式软件开发
7.Web应用开发	MVC/MVP架构、J2EE架构、.NET架构。

计算机系统 校验码

2025年4月21日 2:45

5万

校验码 → 计网

del 体系结构

可靠性 → 信息安全

码距：A码 → B码 需要改变的位数

奇偶校验码：增加1位使1的个数为奇数（奇校验）或偶数（偶校验）

奇校验：含有奇数个1

偶校验：含有偶数个1

只能检1位错，无法纠错

CRC 循环冗余校验码：只能检错 不能纠错

约定一个生成多项式 G(x)，在原始信息位加上若干个校验位

整个信息被G(x)整除（余数为0）则正确

例

1. 在原信息位增加 n-1 个0 10110 0000
2. 由多项式得到除数 最高次幂存在为1 不存在为0
3. 生成 CRC 校验码 进行模2除法运算
4. 将余数添加到原始信息位 生成最终发送信息串
5. 接收方用相同的多项式除，余数为0则无误

海明码：利用奇偶性 在数据位的确定位上插入k个校验位

通过扩大码距 检错和纠错

校验位处于 2^i 上即 1.2.4.8位

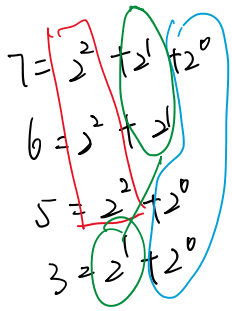
校验位不能超过信息位 如 1010 无第8位校验位

7 6 5 4 3 2 1

计算有几个校验位

计算校验码：信息位拆分 进制表示

计算校验码：信息位拆分 进制表示



4=7⊕6⊕5

2=7⊕6⊕3

1=7⊕5⊕3

原理
编号展开

检错和纠错

将校验码参与信息位的异或　偶校验 为全0
奇校验 为全1

异或的结果=出错的位置.　不可以多位错误

纠错方法为位反转

存储系统

2025年4月21日 星期一 19:28

两级 cache－主存 主存－辅存

局部性原理：时间、空间

存取方式：顺序 直接 随机 相联

性能指标：时间 带宽 周期 传输率

机械硬盘：磁道 扇区 柱面 存取时间：寻道时间（最长时间）+等待旋转时间+读取时间（忽略不计）
（最小单位） 移臂时间

调度算法 先来先服务 FCFS

最短寻道时间 SSTF 饥饿现象

扫描算法（电梯算法）SCAN

单向扫描 CSCAN 头尾往返 与请求无关

廉价磁盘冗余阵列 RAID 多个小磁盘代替一个大磁盘

RAID 0 无冗余无校验

RAID 1 磁盘镜像阵列

RAID 2 纠错海明码 校验盘

RAID 3/4 奇偶校验码 校验盘 利用率 $\frac{n}{n+1}$ $\frac{n}{n-1}$

RAID 5 无独立校验盘的奇偶校验

RAID 6 独立数据盘与2个分布式校验方案

RAID 7 最优化异步IO和高数据传输率 最高持久

RAID 10 高可靠和高性能的组合

Cache 位于CPU与主存之间 由控制部分和存储器组成

Cache 位于 CPU 与主存之间　　由控制部分和存储器组成

地址映射　　主存地址 → Cache 地址　硬件自动完成

直接相联　　全相联映射　　组组相联映射

块号相等　　任意调入　　组之间直接

组内全相联

组相联映射　主存中一个组内页数 = Cache 分组数

替换算法：使 Cache 尽可能高命中

随机替换　先进先出　近期最少使用　优化替换（先验）

命中率　平均时间

写回：写直达　直接同时写 cache 和主存

写回　被淘汰时写回主存

标记法　有效位

输入输出系统 指令 多处理机 可靠性

2025年4月21日 星期一 20:41

输入输出工作方式：程序控制方式（无条件传送 / 程序查询方式）串行

程序中断方式 并行

DMA控制器

通道 统一管理 多路（字节多路通道 低速 / 选择通道 高速 / 数组多路通道 两者结合）

I/O处理机

总线 I/O接口

多处理机 Flynn分类法 指令流——控制部分 / 数据流——处理机

单指单数 SISD 单处理机

单指多数 SIMD 并行·阵列·向量处理机 异步

多指单数 MISD 不可能

多指多数 MIMD 计算机

指令系统：操作码 操作数(地址)

过程：取指 分析 执行

指令寻址方式：顺序 PC 跳跃：本条指令给出

操作数寻址方式：立即 本身

地址 主存

间接 主存地址 → 主存

寄存器 寄存器编号 R_0 R_1 ---

基址

变址

指令系统类型	指令	寻址方式	实现方式	其它
CISC（复杂）	数量多，使用频率差别大，可变长格式	支持多种	微程序控制技术（微码）	研制周期长
RISC（精简）	数量少，使用频率接近，定长格式，大部分为单周期指令，操作寄存器，只有Load/Store操作内存	支持方式少	增加了通用寄存器；硬布线逻辑控制为主；适合采用流水线	优化编译，有效支持高级语言

多处理机（共享存储 / 分布式存储）

分布式存储：海量并行处理结构 MPP；对称多处理机结构 SMP（共享存储多处理机）

本地存储器 LM

均匀存储器存取 UMA → 纯共享存储器

非均匀存储器存取 NUMA → 共享又有本地LM

只用高速缓存 COMA（NUMA特例）共享 + 本地Cache

(共享存储多处理机)

只用高速缓存 COMA (NUMA特例) 共享 + 本地Cache

S2MA (一种NUMA) 共享本地LM本地Cache

可靠性：

平均无故障时间 MTTF

平均故障修复时间 MTTR

平均故障间隔时间 MTBF

串并联可靠性

串联 = $R_1 \times R_2 \times R_3$

并联：$1-(1-R_1)(1-R_2)\cdots(1-R_n)$

1- 都不可靠

N模冗余系统：表决器

操作系统 进程管理 同步互斥

2025年4月22日 星期二 21:11

4个特征 并发性 共享性 虚拟性 不确定性

5个功能 处理机管理 存储器管理 设备管理 文件管理 用户界面

5种分类 批处理(单道、多道),分时(时间片),实时,网络(集中模式 C/S 模式 对等模式),分布式,微型计算机

嵌入式操作系统 主要特点: 初始化

微型化 片级→板级→系统

可定制

实时性

可靠性

易移植性

进程: PCB 程序 数据

三态模型: 运行 就绪 阻塞 (转换)

前趋图 进程资源图(死锁)

进程同步互斥

临界资源: 互斥访问的资源

临界区 : 对临界资源操作的程序

互斥 : 一个任务单独使用

同步 : 多个并发,速度不同

互斥信号量 初值1

同步信号量: 初值为共享资源数

P操作 申请资源 S=S-1

V操作 释放资源 S=S+1

生产者 消费者问题

进程通信 死锁 存储管理 分页分段

2025年4月22日 星期二 21:11

死锁必要条件：1.互斥 2.不剥夺 3.请求与保持 4.环路

解决措施：打破上述条件

方法：死锁预防

死锁避免：提前计算不会死锁才分配

死锁检测：定时检测

死锁解除：强制剥夺 撤消进程

死锁资源计算：n个进程 R个资源 发生死锁最大资源数 $n\times(R-1)$

不发生死锁最小资源数 $n\times(R-1)+1$

线程

分区存储：整存 三种分区时：

固定分区 内部碎片

可变分区 外部碎片

可重定位分区 移动分配地的区域 合并碎片

可变分区算法：

首次适应法：找到第一个满足的

最佳适应法：找到大小最接近的

最差适应法：找到最大的块分割

循环首次适应法：一直往后找第一个满足的

分页存储

逻辑页分为页号和页内地址

优点：利用率高 碎片小 分配及管理简单

缺点：增加系统开销，可能有抖动现象

逻辑地址转换物理地址

页面置换算法：

最优（理论）选择未来最长不被访问的页面

先进先出 FIFO 先调入先淘汰 抖动：分配页数越多缺页可能越多

最近最少使用：LRU 局部性原理 效率高 不抖动

淘汰原则：最近未被访问 未被修改的

快表：小容量 相联存储器 页表副本 按内容访问 速度快 存在Cache

存放当前访问最频繁的少数活动页面的页号

慢表 存在内存中

访问内存 查表(快表/慢表) → 根据页号读取内存

Cache 内存

分段存储 段号 段内地址 段长 基址 按编程逻辑整体分段

段页式 先分段再分页 优点：空间浪费小 存储共享容易 存储保护容易 能动态链接

缺点：实现 开销 硬件 占用内存均增加 执行速度下降

虚拟存储 部分装入主存 其余在磁盘

设备文件作业管理

2025年4月23日 星期三 21:19

设备 输入 输出

分类：数据组织 功能 资源分配 传输速率

功 层次：用户进程
设备无关软件
设备驱动程序
中断处理程序
硬件

Spooling (外围设备联机操作)技术

文件 逻辑结构 { 有结构记录式文件
无结构流式文件

物理设备存放 { 连续结构
链接结构
索引结构 { 直接索引
一级间接索引
二级间接索引
多个物理块索引

计算

文件目录 文件控制块 { 基本信息
存取控制信息
使用信息

相对路径 绝对路径 全文件名

存储空间管理：以物理块 { 顺序存取
随机存取

1. 空闲区表
2. 位示图 Bitmap
3. 空闲块链
4. 成组链接法

文件共享 硬链接 符号链接

保护 存取控制 { 存取控制类矩阵
存取控制表
用户权限表

{ 作业控制级
用户级别或
命令

作业 为一个用户的计算任务求事务处理所做的工作总和 { 程序 数据 作业说明书

{ 脱机 (作业说明书)
联机 (终端)

状态: 提交 后备 执行 完成
(收容)

作业控制块 JCB

调度算法: 先来先服务 短作业优先 优先级调度 响应比高优先

响应比 = 响应时间 / 执行时间

用户界面: 用户与计算机通信的软硬件总称

数据库

2025年4月23日 星期三 22:29

三级模式 内模式 模式 外模式
物理 概念 视图
物理独立性 逻辑独立性

设计：规划 必要性 可行性分析
需求分析 需求说明书 数据流图 数据字典
概念设计 概念模型 E-R图 {属性冲突 命名冲突 结构冲突
逻辑设计 关系模型 逻辑模型
物理设计 物理存储结构和存取方法
数据模型三要素：数据结构 数据操作 数据的约束条件

规划 可行性 必要性分析
需求分析 需求说明书 数据流图 数据字典
概念设计 ER图 {属性冲突 命名冲突 ~~数据~~结构冲突
逻辑设计 逻辑模型
物理设计 存取 方法 物理结构
{数据记录 结构
数据存储安排
选取访问方法设计
安全性 完整性分析
数据库程序设计

E-R图：实体 {强 弱 实体集
属性 属性分类 {简单 复合 单值 多值 NULL 派生 域 名字 取值范围 唯一标识
联系 1对多 1对1 多对多

E-R图 → 关系模型
1个实体 对应一个关系模式
1对1 放到两端任意实体中 作为一个属性
1对多 可以单独作为关系模式 也可以在多端加入1实体作主键
多对多 必须作为一个单独的关系模式 联合主键

关系代数

2025年4月24日 星期四 23:49

并 合并 去重

交 取相同

差 S_1-S_2 S_1有 S_2没有的 S_1减S_2相同的

笛卡尔积 × 属性相加 每一行记录和每一行拼接成为新的一行

投影 Π 选择某列 某属性

选择 选择某条记录 某行

自然连接 展开所有属性列.去重 拼接属性相同的行

笛卡尔积 → 自然连接 连接同名属性列. 选择 同名属性值相同的行

函数依赖 x唯一确定y

部分函数依赖 $ab \to c \Rightarrow a \to c$

传递函数依赖 $a \to b$ $b \to c \Rightarrow a \to c$

公理系统

自反律 $y \subseteq x \subseteq U \Rightarrow x \to y$

增广律 $x \to y \Rightarrow xz \to yz$

传递律 $x \to y$ $y \to z \Rightarrow x \to z$

合并规则 $x \to y$ $x \to z \Rightarrow x \to yz$

伪传递律 $x \to y$ $wy \to z \Rightarrow wx \to z$

分解规则 $x \to y$ $z \subseteq y \Rightarrow x \to z$

z属于y

超键 唯一标识 的属性组合

候选键 超键去冗余 唯一标识的属性组合. 多组

主键 任一候选键

外键 其他表的主键

主属性 候选键 都是主属性. 其他是非主属性

实体完整性约束 主键约束 不为空 不重复

参照完整性约束 外键约束 外键必须是其他表中已存在的主键的值

自定义表达式约束

参照完整性约束　外键约束　外键必须是其他表中已存在的主键的值

用户自定义完整性　自定义表达式约束

范式

1NF　每一个分量不再可分

2NF　每一个非主属性完全依赖任何一个候选码　不存在部分函数依赖

3NF　不存在传递依赖

BCNF　消除主属性对码的部分依赖和传递依赖

4NF　多值依赖　$X \to Y$　$XZ \to Y$　与之无关

如

name	phone 1	phone 2
A	1	2
A	2	1

求候选键：

1. 未在箭头右边的属性　必然是候选键之一　因为不能被其他推导出来
2. 看能否推出遍历所有属性

模式分解 事务并发 封锁协议

2025年4月27日 星期日 22:01

难点

保持函数依赖 每个依赖的左右是否在同一个分解模式中

无损分解 还原出原关系模式

$R \to \rho = \{R_1, R_2\}$

$R_1 \cap R_2 = R_1 - R_2$ 求 $R_2 - R_1$

并发控制

事务 操作 全做 全不做

原子性
一致性
隔离性
持续性

数据不一致

丢失更新
不可重复读
读脏数据

封锁协议

X 排它锁 (写锁) 其他均不可操作

S 共享锁 (读锁) 其他事务可以再加 S 锁

第一级封锁协议

修改前加 X 锁 解决丢失更新

二级封锁协议

读前加 S 锁 读后释放 解决丢失更新 读脏数据

三级封锁协议

读取前加锁 事务结束后释放 解决丢失更新 读脏数据 数据重复读

数据库安全 分布式 数据仓库 数据挖掘

2025年4月27日 星期日 22:33

数据库安全措施

用户标识和鉴权

存取控制

密码存储和传输

视图保护

审计

备份

冷备份

热备份

完全备份

差量备份 完全备份之后的

增量备份 上一次备份之后的

日志文件：用于撤销故障的事务

故障恢复（利用日志）

事务故障 系统自动完成 对用户透明 反向扫描日志文件撤销

系统故障 系统重启时自动完成 无需用户干预 是否提交进行重做或撤销

介质故障 病毒破坏 装入最近副本 重做新提交的数据

检查点 回退到检查点

性能优化

硬件升级

数据库设计 逻辑 增加冗余 分割表（行或列）以进行并行操作

物理 最小存储空间的数据类型 分割关系模式

单独处理文本图片

索引优化 经常查询不更新 索引过多增加维护开销 但可以修改性能

单独处理之外图片

索引优化　经常查询不更新　索引过多虽然加快查询但影响修改性能，

分析重要查询使用频率　少量数据无索引

查询优化　物化视图　减少多表　只检索需要属性

In 代替 OR　经常提交释放锁

分布式数据库

全局DBMS 联网管理所有局部数据库

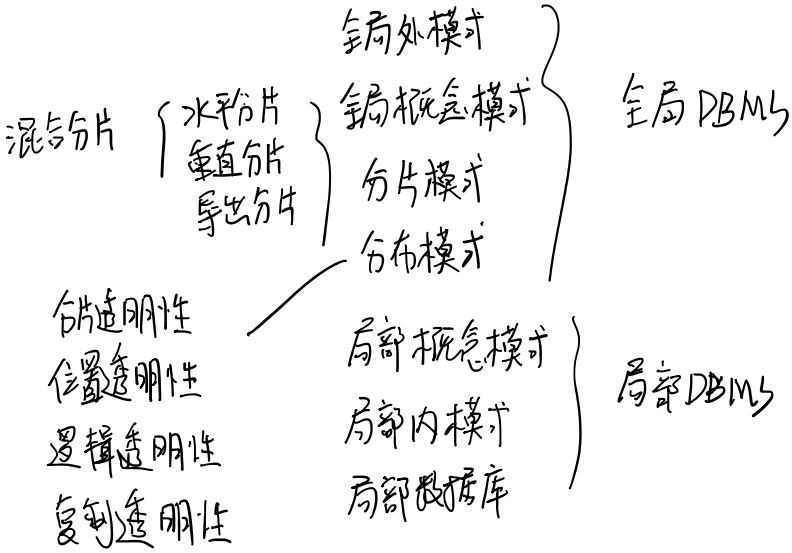
数据独立性

集中与自治共享结合的控制结构

适当增加系统冗余度

全局一致性 可串行性 可恢复性

体系结构



数据仓库

特点：面向主题的　集成的　非易失的　随时间变化的　数据集合.

用于支持管理决策

结构层次

数据源　数据存储与管理　OLAP(联机处理分析)　前端工具

商业智能BI：

主要阶段：数据预处理 建立数据仓库，数据分析 数据展现

↓

ETL

分类：

企业仓库	企业	独立	业务部门
数据集市	部门	从属	中央
虚拟仓库	视图		

设计方法：

自顶向下 企业仓库 → 数据集市

自底向上 数据集市 → 企业仓库

混合法

数据挖掘

发现不能靠直觉发现的 甚至违反直觉的隐藏知识

常用技术 ～

分析方法 ～

非关系数据库 反规范化 SQL语言

2025年4月28日 星期一 21:37

非关系数据库

文档(Json) 键值 Redis 列(数据仓库) 图(社交网络)

反规范化

牺牲规范化提高性能

方法：重新组表，增加冗余列 增加派生列(计算值) 水平垂直分割表

SQL 语言

分组查询 group by 分组附加条件 having

别名查询 select ... as "xx"

插入 insert into .. values( )

删除 delate from

修改 update set

排序 order by

Distinct 去重

union 或

intersect 与

Min AVG Max ...

一般连接 除法 元组演算 第四范式

2025年4月28日 星期一　23:57

一般连接 θ 和等值连接：先笛卡尔积　再满足连接条件

全外连接　自然连接　保留 NULL

左右外连接　保留左、右元组

除运算　R÷S　乘　R中=S的行　去除重复

如 S有两行

R中也要有S的所有行　不完全不行

R

A	B	C	D
a	b	c	d
a	b	e	f
c	d	e	f

S

C	D
c	d
e	f

R÷S

A	B
a	b

只存在一行记录 不完全

元组演算　$\{t \mid R(t)\}$　t 变量　R(t) 公式 条件

$t_{[i]2}$　$t_{[j]1}$　第几个属性

C 常量

计算

元组演算与域演算

A	B	C
1	2	3
4	5	6
7	8	9
10	11	12

R

A	B	C
3	7	11
4	5	6
5	9	13
6	10	14

S

图 7-20　关系 R、S

(1) $R1=\{t \mid R(t) \wedge \neg S(t)\}$ = R-S

(2) $R2=\{t \mid (\exists u)(R(t) \wedge S(u) \wedge t[3]<u[2])\}$

(3) $R3=\{t \mid (\forall u)(R(t) \wedge S(u) \wedge t[3]>u[1])\}$

(4) $R4=\{t \mid (\exists u)(\exists v)(R(u) \wedge S(v) \wedge u[2]>v[1] \wedge t[1]=u[1] \wedge t[2]=v[1] \wedge t[3]=v[3])\}$

A	B	C
1	2	3
7	8	9
10	11	12

(a) R1

A	B	C
1	2	3
4	5	6
7	8	9

(b) R2

A	B	C
7	8	9
10	11	12

(c) R3

R.A	S.A	S.C
4	3	11
4	4	6
7	3	11
7	4	6
7	5	13
7	6	14
10	3	11
10	4	6
10	5	13
10	6	14

(d) R4

35:46 / 45:28

计算机网络

2025年4月29日 星期二 21:30

数据通信

模拟信号 连续

数字信号 离散 码元速率(波特率) $B=\frac{1}{T}$

定理:

奈奎斯特定理(极限速率): 信道带宽为W, 最大码元速率 = $B=2W$

一个码元携带的信息量n位与码元种类数N $n=\log_2 N$ $N=2^n$

离散值个数

R 数据速率 = $B\log_2 N = 2W\log_2 N$

香农定理(有噪声极限速率): $C=W\log_2(1+\frac{S}{N})$

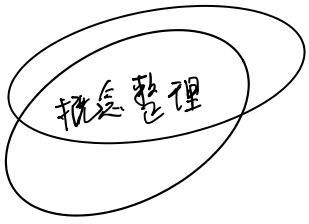
带宽 信噪比

分贝转换 $dB=10\log_{10}\frac{S}{N}$

数字→模拟 调幅 调频 调相

AM FM PM

数字 重新编码



分类

通信子网 局域网

资源子网 广域网 公共 专用 无线

拓扑 总线型 星型 环型 树型 分布式 网状

以太网协议 IEEE 802.3

采用带冲突检测的载波监听多路访问 CSMA/CD

人载波监听

2.冲突检测　边发边听的冲突检测

无线局域网　IEEE 802.11

拓扑　点对点　HUB　完全分布型

广域网特点：

面向数据通信的服务　远距离

范围广　距离远

电信部门　企业　免费　有偿服务

相关技术：

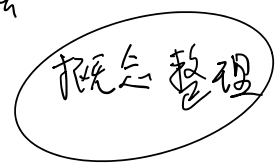
同步光网络 SONET

数字数据网 DDN

帧中继　FR　X.25

异步传输模式 ATM

OSI 七层



TCP/IP

数据安全 网络安全 访问控制

2025年5月3日 星期六 15:44

常用身份认证 口令认证 动态口令认证 生物特征识别
(数字签名)

密钥管理

KMI： 密钥分发中心（KDC） 静态分发 { 点对点分发 / 一对多分发 / 网格状分发 } 需要秘密信道
动态分发 { 对称加密分发 / 非对称加密分发 }

PKI： CA 适用 所有网络应用 不需要秘密信道

SPK机制： 多重公钥
组合公钥

网络安全：

防火墙 功能
分类：网络级 应用级
详细分类：包过滤型 电路级网关型 应用网关型 代理服务型 状态检测型 自适应代理型
体系结构：堡主机模式 屏蔽主机模式 屏蔽子网模式
局限性

VPN：

SSL 传输层

HTTPS 443

PGP 邮件 RSA

IPSec 二业

单点登陆
SSO
Kerberos KDC 对称加密
外壳脚本

访问控制技术 主体 客体 控制策略
模型 BLP 下读 上写 保密性
Biba 上读 下写 完整性

Lattice

分类
- 自主 DAC
- 强制 MAC
- 角色 RBAC
- 任务 TBAC 实时
- 对象 OBAC

容灾 安全管理 可靠性 冗余容错

2025年5月3日 星期六 17:12

容灾 容备 灾恢 灾恢等级 1-6

RPO RTO

级 数据容灾 应用容灾

灾恢规划 风险分析 业务影响评估 确定灾恢目标

灾恢策略制定 7要素

实现

驱案

业务持续性规划 BCP

安全管理体系：

密码管理
网络管理
设备管理
人员管理

安全审计

控制目标
安全漏洞
控制措施
控制测试

CC标准

可靠性

冗余技术 结构冗余

信息冗余
时间冗余
冗余附加

软件容错技术 N版本程序设计 静态 前向恢复

恢复块 动态 后向恢复

防卫式程序设计

信息资源管理 企业信息化规划 ERP

2025年5月3日 星期六 17:57

IRP 信息资源规划　　需求分析　系统建模

业务

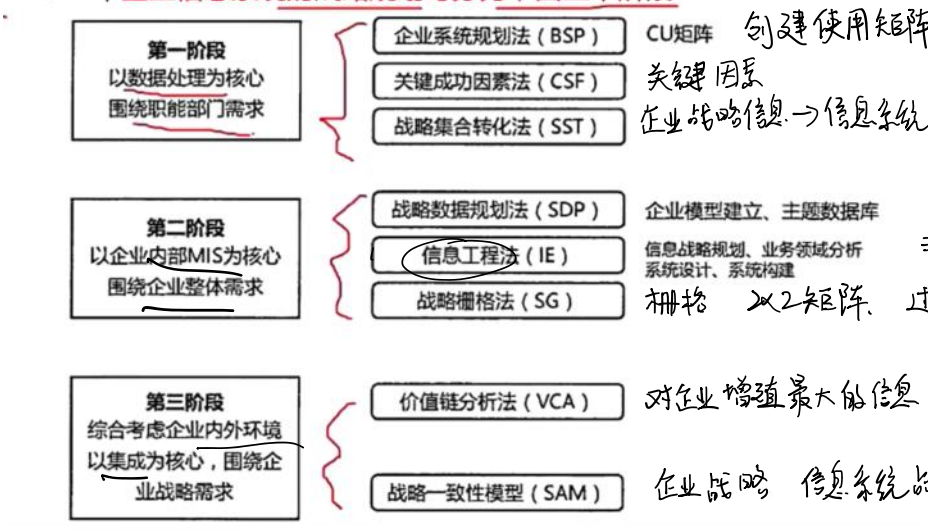
数据

企业战略与信息化战略集成主要方法：

业务与IT整合

企业IT架构

◆一个企业信息系统的战略规划可分为下面三个阶段：



创建使用矩阵　自上而下分析　自下而上设计

关键因素

企业战略信息→信息系统战略信息

三要素：企业各种信息　业务过程　采用的信息技术

2×2矩阵　过程　数据

对企业增值最大的信息

企业战略　信息系统战略　一致

ERP 企业资源规划　　供应链所有环节进行有效管理

物流资金流　信息流

管理思想　软件产品　管理系统

主要功能　财会　物流　生产控制　人力资源

技术：二次开发　定制开发

企业信息系统 企业应用集成

2025年5月3日 星期六 20:02

CRM 客户关系管理 以客户为中心 → 帮助企业提高获取利润的能力

模块 销售自动化 营销自动化 客户服务与支持 商业智能

功能 客户服务 市场营销 共享客户资料 分析能力 } 支柱性功能

CRM解决方案：

畅通有效的客户交流渠道 触发中心

对所获信息进行有效分析 挖掘中心

与ERP 很好地集成

供应链 SCM

PDM 产品数据管理

产品生命周期PLM 5阶段

决策支持系统 DSS

知识

企业门户

电子商务

电子政务

企业信息集成

1. 表示集成 黑盒集成 统一入口
2. 数据集成 白盒集成 统一数据格式 中间件
3. 控制集成 黑盒集成 业务逻辑层次
4. 业务流程集成 最价值最综合的集成

少企业间集成

事件驱动架构　订阅/发布

工作流

软件工程

2025年5月4日 星期日 17:38

软件生命周期：基本过程　支持过程　组织过程

软件 能力成熟度模型 CMM：初始级　杂乱　仗势

可重复级　基本的项目管理过程

已定义级　文档化 标准化　标准 软件过程

可管理级　详细度量标准

优化级　持续优化

软件 能力成熟度模型集成 CMMI：

组织成熟度

阶段式：

可重复级

已定义级

可管理级

优化级

连续式：

表 7-2　阶段式模型的过程域分组

成熟度等级	过程域
可重复级	需求管理、项目计划、配置管理、项目监督与控制、供应商合同管理、度量和分析、过程和产品质量保证
已定义级	需求开发、技术解决方案、产品集成、验证、确认、组织级过程焦点、组织级过程定义、组织级培训、集成项目管理、风险管理、集成化的团队、决策分析和解决方案、组织级集成环境
可管理级	组织级过程性能、定量项目管理
优化级	组织级改革与实施、因果分析和解决方案

软件过程 评估：

CMM：软件组织

Trillium：嵌入式 软件开发

Bootstrap：改进 CMM　质量属性

ISO/IEC 15504

中国行业标准 《软件过程能力评估模型（SJ/T 11234-2001）》

软件过程 模型：

瀑布模型：需求明确

螺旋模型：高风险　（瀑布 + 快速原型）

V模型：测试

原型模型　需求不明确

增量模型

喷泉模型：面向对象

CBSD 基于构件的开发模型

形式化方法：数学化　　增量软件工程 CSE

结构化开发方法：

结构化设计　结构化程序设计

结构化开发方法：

结构化分析　结构化设计　结构化程序设计

主要特点：开发目标清晰化　用户第一

开发工作阶段化

开发文档规范化

设计方法结构化　自顶向下分析设计　自底向上实现　+

不足：开发周期长

不适合需求不明确

面向过程　面向数据流不考虑数据结构

常用工具：数据流图　数据字典　结构化语言　判定表　判定树

面向对象方法：普遍适合各类系统开发

OO方法

不足：依靠面向对象技术支持

不能涉足系统分析以前的开发环节

快速原型法：适合需求不明确，分析难度大　技术难度不大　有效确认用户需求

开发思想

是否实现功能：水平　垂直

最终结果：抛弃式　演化式

特点：开发周期缩短

成本风险降低

速度加快

获得较高的综合开发效益

以用户为中心

不足：开发环境要求高

管理水平要求高

面向服务：降接口定义和实现解耦

操作　单个逻辑单元

服务　操作的逻辑分组

业务流程　最高层

快速应用开发RAD：基于构件的开发方法

开发阶段：业务建模
数据建模
过程建模
应用生成
测试与交付

局限：不适用所有应用
很短时间完成需求分析
不适用技术风险高的项目

敏捷模型：

宣言：个体和交互胜过过程和工具
可以工作的软件胜过面面俱到的文档
客户合作胜过合同谈判
响应变化胜过遵循计划

核心思想：适应性
以人为本
迭代增量式

主要方法：极限编程 XP 、螺旋式 测试先行
动态系统开发方法 DSDM 以业务为核心
并列多球法 Scrum ：多个短的迭代同期 Sprint 并行

统一过程模型

RUP四个阶段：初始
细化
构建
移交

特点：用例驱动
以体系结构为中心 4+1 用例视图
迭代与增量

软件开发环境和工具

含义：按开发模型与开发方法：瀑布 演化 螺旋 喷泉… 面向对象 面向服务 …
按功能与结构特点 单体型 协同型 分散型 并发型
按应用范围 通用型 专用型
按开发阶段 前端 后端 维护 逆向

集成机制：环境信息库 核心
过程控制与消息服务器

环境任务集

过程控制与消息服务器

环境用户界面

工具集：开发

维护

管理支持

RAD工具：可视化

软件复用：

逆向工程

1)重构:通过调整程序代码改善软件的质量、性能，使其程序的设计模式和架构更趋合理，提高软件的扩展性和维护性。

2)设计恢复:借助工具从已有程序中抽象出数据设计、总体结构设计和过程设计等信息。

3)再工程：在逆向工程所获得信息的基础上，修改或重构已有的系统，产生系统的一个新版本。

4)正向工程:不仅从现有系统中恢复设计信息，而且使用该信息去改变或重构现有系统，以改善其整体质量。

软件产品线：双生命周期模型 { 软件工程 / 应用工程

SEI 模型 {

三生命周期模型

建立方式：演化 革命

现有

全新

形式化方法

项目管理

2025年5月5日 星期一 19:08

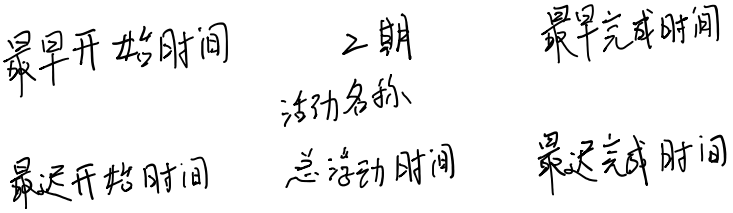
范围管理

进度管理

活动排序

活动资源估算

活动历时估算



顺推：最早开始时间 ES = 所有前置活动最早完成 EF 的最大值

最早完成 EF = 最早开始 + 持续时间

逆推：最晚完成 LF = 所有后续活动最晚开始 LS 的最小值

最晚开始 LS = 最晚完成 - 持续时间

总浮动时间：最迟开始 LS - 最早开始 ES

最迟完成 LF - 最早完成 EF

关键路径 - 非关键路径

自由浮动时间 = 紧后活动的最早开始时间的最小值 - 本活动的最早完成时间

预算和成本管理

配置管理

质量管理

人力资源管理

风险管理

系统规划与分析

2025年5月6日 星期二 02:32

可行性分析 {必要性 / 可能性

经济可行性（消线） 技术可行性 法律可行性 用户使用可行性

盈亏平衡点：销售额 − 成本（固定+可变） = 利润

= 总固定成本 + 总可变成本

= 总固定成本 + （可变成本占销售额的比例 × 盈亏平衡点销售额）

净现值分析：考虑货币的时间价值

投资收益率：（折现收益 − 折现成本）/折现成本

投资回收期：收支平衡点

系统分析

问题分析 1.研究问题的领域 上下文图

2.分析问题和机会

3.制定系统改进目标

4.汇报调查结果与建议

业务流程分析

方法 价值链分析法：使客户满意、实现客户价值最大化的业务流程

客户关系分析法：客户与企业的关系

供应链分析法：供应链核心环节

基于ERP分析法：供应链所有环节进行管理

业务流程重组：优势环节 客户满意为出发点

工具：业务流程图 TFD 业务活动图 BAM 统一建模语言 UML 的活动图

业务流程建模

企业业务流程 三要素 实体 对象 活动

建模方法：

标杆瞄准 确定出代表最佳实践的经营过程和工作过程，以便合理地确定本企业的业务流程。 复制优秀企业的成功经验

IDEF

Petri网 图形化 数学化 适合描述并行、异步、分布式和随机性 的信息系统

DEMO 定义了信息系统中行为角色之间的通信方式 理论基础是对话行为理论 核心是业务事务 发起者 执行者 要求阶段执行阶段 结果阶段

业务流程建模语言 文本类 图元类 BPEL BPML XPDL BPMN UML

数据与数据流程的分析

系统方案的建议

软件需求

2025年5月6日 20:43

需求开发
- 需求获取
- 需求分析
- 需求定义（需求规格说明书）
- 需求验证

需求管理
- 需求基线
- 变更控制
- 版本控制
- 需求跟踪
- 需求状态跟踪

需求分类：
- 业务需求
- 用户需求
- 系统需求
  - 功能需求
  - 非功能需求
  - 设计约束

软件需求 FURPS+

需求获取

方法 优缺点 适用

- 用户访谈 代表性用户 结构化 非结构化
- 问卷
- 采样 数理统计
- 情节串联板
- 联合需求计划 JRP 关键用户代表 开会
- 需求记录技术

需求分析 用户要求 期望 转化为用户需求

PDOA 面向问题域的分析方法 强调描述 更基于建模

SA 结构化方法 特点：自顶向下 逐步分解 面向数据

三大模型：
- 功能模型 数据流图
- 行为模型 状态转换图 STD
- 数据模型 E-R图

需求分析的任务

(1)绘制系统上下文范围关系图
(2)创建用户界面原型
(3)分析需求的可行性
(4)确定需求的优先级
(5)为需求建立模型
(6)创建数据字典
(7)使用QFD(质量功能部署)

(核心) 数据字典

OOA 面向对象的方法

对象 名 属性 方法(行为)

类 对象是类的实例 类是对象的抽象 模板 实体类 控制类 接口类

抽象 抽取共同特征 强调主要忽略次要

封装 信息隐藏

继承 子类继承父类

多态 参数 包含 重载(过载)

接口

消息 对象交互

覆盖 父类子类 替换

函数重载 函数名相同 参数不同

绑定 静态 动态

建模: 分析模型

用例模型

需求定义 平均定义方法 原型方法

需求验证

需求评审: 评审 检查 走查

技术评审: 正式 非正式

需求测试: 概念测试用例

需求管理 需求变更 需求变更控制委员会 CCB

需求跟踪 正向 逆向 需求跟踪矩阵

软件架构设计

2025年5月10日 星期六 14:02

架构风格

质量属性

发展阶段

软件架构为软件系统提供了一个结构、行为和属性的高级抽象，由构件的描述、构件的相互作用(连接件)、指导构件集成的模式以及这些模式的约束组成。软件架构指定了系统的组织结构和拓扑结构，并且显示了系统需求和构件之间的对应关系，提供了一些设计决策的基本原理。

无架构阶段 汇编语言

萌芽阶段： 控制流图 数据流图 结构化阶段

初级阶段 UML 面向对象

高级阶段 4+1 模型

模型：
- 结构
- 框架
- 动态
- 过程
- 功能

4+1视图模型

逻辑视图	功能需求		
开发视图	UML实现视图	内部需求	
进程视图	运行特性	非功能需求	
物理视图	UML 部署视图	软硬件映射	
场景	重要系统活动	UML 用例视图	

软件架构风格、早期编译器

我语法体系结构风格 { 批处理 / 管道-过滤器

前一个输出作为后一个输入 前面执行部分下一个就可以开始

自一步必须在前一步结束后开始 数据传递短 整体串行

调用-返回体系结构风格 { 主程序/子程序 / 面向对象 / 层次结构

把问题划分若干步骤 过程调用作为交互机制完善连接件

通过函数和过程的调用交互

连接件 两层间交互协议 定义 修改一层最多影响2层

优点：将复杂问题分解成增量步骤序列的实现 越靠近底层 抽象级别越高 为软件复用提供支持

缺点：不一定能划分 即不易划分

两层C/S 客户端 服务器 都有处理功能 现在不常用

三层C/S 将处理功能独立

优点：逻辑独立 并行开发 灵活选择硬件部署 系统安全管理可控

缺点：通信效率较低

- 表示层 客户机
- 功能层 应用服务器
- 数据层 数据服务器

B/S 浏览器 0客户端

连接件是消息传递 方式：点对点 同步异步 RPC

独立构件风格 { 进程通信 / 事件驱动系统(隐式调用)

解释器 { 解释引擎 存储区 伪状态和执行进度的数据结构

基于规则的系统 { 规则集 解释器 选择器 工作内存 用户接口 智能纵横和DSS

发布或发送广播一个或多个事件，事件触发后系统自动调用事件注册的所有过程

优点：软件复用 方便构件维护演化

缺点：放弃对系统计算的控制

现代编译器 以数据为中心 { 仓库架构风格 / 黑板架构风格

仓库是存储维护数据的重要场所 两种构件：中央数据结构 对中央数据进行操作的独立构件

解决复杂非结构化问题 知识源 黑板 控制 知识源提供知识 响应 黑板变化 黑板是全局数据库

面向服务的架构风格：明确定义的接口 主要是 WSDL

自包含模块化

粗粒度 松耦合

互操作性 策略声明

相关技术

UDDI 统一描述 发现和集成 Web 服务注册和查找

① 数据模型

② API

③ 注册服务

WSDL Web服务描述语言 服务采取定义服务接口

SOAP 简单对象访问协议 { 信封 } 服务请求者和服务提供者之间消息传输载体 XML 格式化

WSDL Web服务描述语言　　服务实现定义 服务接口定义

SOAP 简单对象访问协议　　服务请求者 和 服务提供者之间 消息传输机制　XML 格式化

① 封装　信封 { SOAP头　SOAP体
② 编码规则
③ RPC表示
④ 绑定

REST 表述性状态转移　　只用 HTTP 和 XML 通信

降低开发复杂性　提高系统可伸缩性

① 所有事物抽象为资源
② 唯一资源标识
③ 通用连接件操作对资源进行操作
④ 操作不改变资源标识
⑤ 操作无状态

实现方法

1. Web Service

服务提供者 服务请求者 注册中心

发布查找绑定

层次：底层传输层
服务通信协议层
服务描述层
服务层
业务流程层
服务注册层

2. 服务注册表

存储 服务和相关构件的配置、依从性和约束文件

服务注册
服务位置
服务绑定

3. 企业服务总线 ESB

消除提供者和请求者直接连接　　解耦

1. 异构环境中 服务 消息 基于事件的交互
2. 不更改代码 无缝 非侵入 方式
3. 充当缓冲器 ESB 不同系统同时使用一个服务
4. 服务代理 协议转换
5. 消息转换 翻译机制 基于消息内容的消息路由服务
6. 安全机制 拥有者机制

优势：扩展性 基于标准的连接
灵活的 服务导向的应用组合
提高高复用率 降低成本
减少市场反应时间 提高生产率

软件架构标准

IEEE 1471-2000

软件架构标准要素之间的关系　5个层次

任务
环境、系统及架构
利益相关者 架构说明及原理
关注 关注点 视图
关注点库 模型

基于架构的软件开发

模型开发过程：

软件结构
- 需求：需求获取 — 系统质量目标 系统商业目标 开发人员商业目标；标识构件 — 为系统生成初始逻辑结构
- 设计：将需求映射的标识构件映射成构件
- 文档化：获取说明　质量设计说明书
- 复审：外部人员复审
- 实现：实体显示架构　实现构件　组装系统
- 演化：改变架构　增删构件　使架构可复用

软件架构质量属性：
- 开发期质量属性　软件开发阶段
- 运行期质量属性　软件运行阶段

软件架构评估质量属性：

性能：系统响应能力　如响应时间 吞吐量
设计策略：优先级队列　增加计算资源　减少计算开销　引入并发机制　采用资源调度

可靠性　在意外或错误使用 情况下维持软件系统的功能特性　#如 MTTF　MTBF　MTTR

可用性　设计策略：心跳　Ping Echo　冗余 选举
能够正常运行的时间比例　#如故障间隔时间

安全性　系统在向合法用户提供服务的同时阻止非授权用户使用的企图或拒绝服务的能力
如保密性　完整性　不可抵赖性　可控性
设计策略　入侵检测　用户认证　用户授权　追踪审计

可修改性　以较高的性能价格比 对系统进行变更
设计策略：操心一类似合类　抽象　信息隐藏

功能性　系统所能完成 所期望工作的能力

可变性　体系结构 经扩充或变更成为新体系结构的能力

互操作性　组成部分的软件之间 独立存在的，经常与其他系统或自身环境 相互作用

质量属性场景：面向特定质量属性的需求

刺激源
刺激
环境
制品
响应
响应度量

系统设计

2025年5月10日 星期六 21:57

概要设计 形成模块结构图 即系统结构图

详细设计

网络设计 选择网络结构

代码设计 编码问题 分类 {线分类法 面分类法

输入输出设计 关键性正确性 一致性

处理流程设计 目的 考虑模块的内部结构 即内部执行过程

数据存储设计 选择数据存储的方式 存储介质 数据组织方式 和记录格式

包括：数据的级着安排

文件或数据结构设计

用户界面设计 三大原则 {置于用户控制之下 减轻用户的记忆负担 保持界面一致性

软件性和可靠性设计 对系统运行环境 软件生存分析控制

包括对数据处理的控制

处理流程设计：

流程设计工具 {图形 程序流程图 PFD 顺序型 选择型 多选选择型 当型循环 直到型循环

IPO图 输入 输出 数据加工

NS图(盒图) 嵌套和层次关系 不适合复杂结构的设计

问题树 结构化程序设计

判定树 逻辑判断问题

表格 判定表 逻辑判断问题

语言 过程设计语言 结构化语言 伪代码 描述处理过程 外层描述结构 内层描述操作

6章节 业务流程重组 BPR 根本性的再思考 和彻底性的再设计

分类 管理流程

操作流程

支持流程

实施方法：

1. 在研究和描述企业现有的 业务流程的基础上重新设计

2. 以 O 开始

设计原则：以流程为中心

团队管理原则(以人为本)

以顾客为导向

步骤：战略规划

流程规划

数据规划

功能规划

实施规划

面向结构

内聚：

分类 偶然内聚 无直接关联

逻辑内聚 若干个逻辑相似的功能 通过参数确定哪一个功能

时间内聚 同时执行

过程内聚 指定过程顺序

通信内聚：相同数据结构 相同输入输出

顺序内聚：顺序执行 前一个输出为下一个输入

功能内聚 共同作用 缺一不可

耦合分类：

无直接耦合 无关系

数据耦合 传递函数或调用

标记耦合 传递数据结构

控制耦合 传递控制变量 选择执行某一功能

外部耦合 软件外部环境

公共耦合 公共的数据结构 全局变量

内容耦合 内部关联

系统结构图：模块结构图： 反映功能关系和模块间的联系与通信 包括各模块的层次结构 即反映系统的总体结构

特殊类型模块：传入 传出 变换 协调

3种类型 事务型 变换型 混合型

面向对象

、 、 、 、 (继承和多态)

系统设计基本原理

抽象化;

自顶而下,逐步求精;

信息隐蔽;

模块独立(高内聚,低耦合)。

系统设计原则

保持模块的大小适中;

尽可能减少调用的深度;

多扇入,少扇出;

单入口,单出口;

模块的作用域应该在模块之内;

功能应该是可预测的。

弱类型　强模型　静态型　混合型

面向对象

基本思想：抽象　封装　可扩展性（继承和多态）

对象持久化　内存中的对象保存到数据库/存储设备中　对象/关系 映射

数据持久化：封装数据访问细节　API　减少访问数据库次数　代码重用性高　批数都高

主流框架　CMP　Hibernate　iBatis　JDO

设计原则

开闭—封闭　可扩展　不可修改

里氏替换原则　3类型　能够替换基类型

依赖倒置原则：抽象不依赖细节　细节依赖抽象

组合/聚合复用原则　合成复用原则　尽量使用组合聚合，少用继承

has A 聚合/组合　is A 继承

接口隔离原则　用多个专门接口　而不是单一接口

最少知识原则 迪米特法则　尽量少与其他实体发生相互作用

做到　低耦合通信就不应有相互作用　必要就通过第三者转发

以　对象间信息流量　流向　信息影响最小到

设计模式：

四要素 名称 问题 解决方案 效果

△ 分类：

创建型

Factory Method 工厂方法模式：　3类决定到底创建哪一个类

Abstract Factory 抽象工厂模式：　提供抽象接口 无需指定具体类

Prototype 原型模式：　创建这个原型对象　深克隆 浅克隆　接口

Singleton 单例模式：　唯一实例

Builder 建造者模式：　构建和表示相分离，同一个构建可以创造出不同的表示

结构型：

Adapter 适配器模式：　转换 兼容接口

Bridge 桥接模式：　抽象和实现分离

Composite 组合模式：　整体—部分 树型结构

Decorator 装饰模式：　附加职责

Facade 外观模式：　统一对外接口

Flyweight 享元模式：　共享技术 细粒度对象

Proxy 代理模式：　代理 控制引用

行为型

Chain of Responsibility 职责链模式：　链接 传递请求 职责

Command 命令模式：　请求 参数化 可撤消

Interpreter 解释器模式：　解释器 虚拟机

Iterator 迭代器模式：　遍历访问而不暴露内部表示

Mediator 中介者模式：　中介对象 不直接引用　ESB 企业服务总线

Memento 备忘录模式：　状态恢复

发布-订阅模式 observer 观察者模式：　状态改变 一通知 一自动更新

State 状态模式：　状态改变 — 行为改变　似乎修改类

Strategy 策略模式：　算法替换

Template 模板方法模式：　模板

Visitor 访问者模式：　数据和操作分离

输入输出原型设计

人机交互设计

软件实现与测试

2025年5月12日 星期一　23:05

实现：

- 结构化：自顶向下 逐步求精　单入口 单出口　先整体后局部 先抽象后具体
  - 经典：顺序 选择　Do while
  - 经典+扩展：多分支　Do until
  - 经典+扩展+修正：Leave　Break
- 面向对象 OOP：问题域　对象之间的关系　提高可靠性 可维护性 可复用性
- 可视化：　控件

程序语言的选择：

理想的模块化机制
可读性好的控制结构和数据结构
编译程序能够尽可能多地发现程序中的错误
独立编译机制

程序设计风格：

可理解性
可复用性
可扩展性
健壮性

编码规范　命名规范
注释原则
编排风格原则
版本管理规则

代码生成：

软件重用三个层次：代码重用：复用函数 对象 模块 类 算法
设计结果重用
分析结果重用

软件测试：人工 自动 运行 测量 软件 检验 软件 满足 需求

基本问题　验证功能 实现 和 可用性　检查程序的错误

测试对象：源程序/目标代码　各阶段的开发文档 配置数据

目的：
用户
开发者
测试人员

原则：预期输出和结果能对应
开始避免测试自己的程序
组织避免测试自己的软件
检查每一个测试结果
不应默认假定不会发现错误
不要丢弃
尽早执行测试
追溯到需求

测试方法：（是否可以分类）

黑盒测试　功能测试　不要求内部逻辑结构　测试界面与功能

设计新的类尽可能多地覆盖尚未被覆盖的有效等价类

测试方法：

黑盒测试　功能测试　不考虑内部逻辑结构　测试界面与功能

方法：等价类划分法 1.设计新的类尽可能多地覆盖 尚未被覆盖的有效等价类

2.设计新的测试用例 使其仅覆盖一个尚未覆盖的有效等价类

、边界值分析法 范围的两端 范围之外的极限值 如(0,150)

例如 -1,0, 150.151)

、错误推测法.因果图法.判定表驱动法

正交试验设计法 功能图法 场景法

白盒测试：结构测试 了解程序内部逻辑结构 所有路径进行用例覆盖

方法：代码检查法 静态结构分析法 静态结构度量法 逻辑覆盖法

基本路径测试法. 域测试 符号测试 路径覆盖 程序变异

覆盖标准：逻辑覆盖，

语句覆盖SC 每条语句至少执行一次　覆盖层级最低

判定覆盖DC 每个判定的每个结果至少执行一次　真假结果都执行一次

条件覆盖CC 每个判定的每个条件应取到各种可能的值　每个子条件都判断真和假

判定/条件覆盖 同时满足判定和条件覆盖

条件组合覆盖 各条件的每一种组合都至少出现一次

修改条件判断覆盖 每条件可以独立影响判断结果

循环覆盖

基本路径测试　所有路径均覆盖　层级最高　所有组合都走完

函数

语法

灰盒　介于白盒黑盒　集成测试　表征性反映内部运行情况

软件度量：Mc Cabe度量法 环路复杂度 以程序流程图

计算公式：判定节点个数+1

测试方法：(是否需要执行代码)

静态测试：不运行　更有效率

代码检查：代码走查　程序员代替计算机 手动运行测试用例

桌面检查　程序员自查

代码审查　开会评审

静态结构分析

代码质量度量

动态测试：运行　检查运行结果和预期

构造测试用例

执行程序

分析输出结果

自动化：接口 UI 单元

测试类型：

测试对象：

功能

性能：负载测试 满载时测试

压力测试 从0加压直到某一项满载崩溃

并发测试：负载和压力结合 直到崩溃或不能接受的性能点 满载继续加压

容量测试：某项指标的极限值

可靠性测试：满足可靠性要求

安全 应用本身的安全 数据安全

兼容性 软件之间是否会相互影响

界面 布局是否合理

易用性 用户体验测试

稳定性 长时间运行

测试阶段：

单元测试 最小可测试单元 最低级别 白盒 开发人员

集成测试 集成起来后测试 高层设计 模块、子系统、系统集成

系统测试 恢复测试

安全测试

压力测试

验收测试：

正式

非正式 主观

Beta 多个用户实际使用

被测软件：

APP

Web

物联网

车联网

大数据 数据测试 大数据系统和大数据应用产品测试

AI

小程序

其他分类：

回归测试 修改旧代码 测试没有引入新的错误

测试效率和有效性策略 再测试全部用例

基于风险选择用例

基于操作剖面选择测试

基于风险选择出测
基于操作剖面选择测试
再测试修改的部分
冒烟测试 嵌入产品前
形成集成测试版本前
形成集成测试版本
后期须测试缺陷修正

测试组织
项目需求分析阶段 制订测试计划阶段 搭建测试环境阶段
执行测试阶段 缺陷管理

配置管理对象：软件测试计划 测试方案 测试版本 测试用例 环境 测试结果

过程管理：
基本内容：项目启动 测试计划 测试设计 测试执行 测试结果审查和分析阶段

缺陷管理：确定优先级
同步缺陷状态

风险管理：
风险识别
风险评估
风险应对
风险上报
风险规避
风险转移
风险减轻
风险接受

软件部署：

系统运维

2025年5月13日 星期二 23:47

MTTR 平均故障修复时间
MTBF 平均故障间隔时间
MTTF 平均无故障时间
MTTA 平均应答时间

系统运行管理

系统用户管理
统一用户管理
身份认证方法
用户安全审计

网络资源管理
网络管理
管理系统

软件管理
构件管理
分发管理
文档管理

故障管理 影响度 紧迫度 优先级
故障监视
故障调查
故障排查 恢复处理
硬件 备份 进程
数据库 日志
应用软件 调试

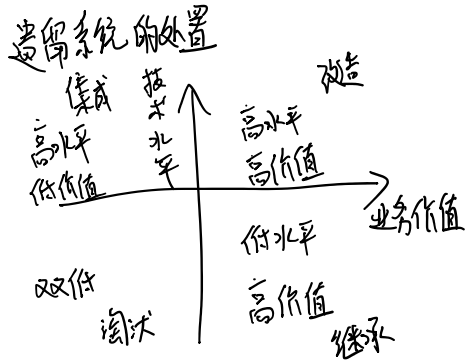
故障收尾

软件维护

维护性：模块化 可重用性 易分析性 易修改性 易测试性 维护的经济性
分类：改正性维护 适应性维护 完善性维护 预防性维护

系统评价

系统改进建议



系统转换

直接转换

并行转换

分段转换

数据的转换和迁移

方法

准备工作

实施 扫描 转换 装载

检验

Web应用系统分析与设计

2025年5月19日 星期一 00:38

常见优势 易于访问开发高效 使用简单 扩展性强

2种通用构建方法： 1. 在服务器 执行大部分应用逻辑

特征： 云就绪和可扩展性 跨平台 自动化测试 支持传统应用和SPA 简单开发和部署

通用设计原则 分离关注点 根据工作类型分离

封装 隔离其他部分

依赖关系反转 依赖方向 为抽象的方向

显式依赖关系 显式 指明所需依赖

单一责任 适用面向对象设计 一个对象只有一个责任

避免自我重复 避免多个位置指定与特定 概念 相关的行为

接口性无知 保持不变的类型 但代码不变 持久性技术的影响

有界上下文 将大型应用程序分解 独立的概念 模块 解决复杂性问题

开发部署模式分类： 整体式 传统 整个软件一般代码 瀑布模型 所有组件相互依赖关联
不适合大型复杂项目 单点故障 难扩展

微服务 松散耦合的独立服务 可扩展 灵活 适合复杂 管理较复杂

无服务 基础结构由 服务商提供 无需为空闲 CPU 付费

架构模式：B/S 三层 表现层 逻辑层 数据层

P2P 客户端直接通信 { 每个客户端之间都有连接
环形拓扑

MVC/MVP 架构 模型 视图 控制器 Presenter 负责逻辑处理 将 视图与模型分离开

微服务 单一应用程序构成的小服务 轻量化 全自动部署 不同语言 数据库等组件实现 高并发高性能高可用 存在服务管理问题

框架： 一组 组件能集合 重用架构知识的时

Java EE

SSH { Spring 解决复杂性 分层架构可选择组件 整合各类框架 统一配置部署 运行期间配置所需资源
Struts 基于MVC 采用 Java Servlet JSP XML 实现
Hibernate 开源对象关系映射 框架 对 JDBC 进行非常轻量级对象封装

JSF 构建 web 的标准 Java 框架

.NET 敏捷软件开发 快速应用开发

Web层

Web Page 3.0 MVC 重点关注 view 可视化开发 最大限度重用

AJAX

开发 选择 编码 内容创建 集成重构 以及测试活动

通信协议

HTTP

RTP/RTSP 音视频 应用层 一对多 多媒体

SMTP/POP3 邮件

数据存储

缓存系统 本地 频繁数据 请求 从缓存中读取

1. 应用程序服务器缓存
2. 全局缓存
3. 分布式缓存
4. 内容分发网络 CDN

1. 应用静态服务器预处理
2. 全局缓存
3. 分布式缓存
4. 内容交付网络 CDN

云存储　集群应用　网格技术　分布式文件系统
CDN 内容交付网络　负载均衡　TCP/IP　应用级　软件/硬件
负载均衡　分布到多个服务器
负载均衡　物理位置、路由到更近的

消息队列　异步存储　web 不同服务间通信

客户端技术　展现内容和交互
HTML5　标准化
XML　处理结构化信息　分布式架构 信息交换
DHTML　动态更新 web 页面内容
CSS　层叠样式表　为结构化文档增加样式
Flash/Flex　动画、矢量图 体积小 边下边播 优化好 保护 禁止下载
Dom　标准编程接口　Web 页面与 Script 的桥梁
Js/AJAX　脚本语言　异步交互　无需刷新

服务器技术
CGI　运行服务器端外部程序
PHP　HTML 内嵌式脚本语言　更快
ASP/ASP.NET　动态 HTML 内嵌　通用　内置对象增强脚本功能　ActiveX 组件 代码不可见
Servlet　Java　控制器　Java 优点 抽象级别高　控制器功能强
JSP　高层 Servlet
Perl/Ruby/Python　面向对象 解释型脚本语言

部署：　打包 注册 发布
部署粒度　非常细粒度　最好一组变化 一起打包　提供一项功能和特性的 Web 应用增量
部署原则　管理用户期望　安装测试交付包　文件前向兼容支持制度　改正缺陷后交付
部署环境　操作系统软件　Web 应用服务器　解决 负载均衡 数据库连接池 高速缓存机制
部署版本控制/CMS　变更管理工具

测试：　发现错误和缺陷
测试内容：功能测试　内容测试　性能测试　web 页面测试　客户端兼容性测试　安全性测试

设计阶段测试的主要任务是：估算服务器端容量的规划是否合理，系统的安全设计是否合理，数据库设计是否合理，检查客户端设计的功能是否正确合理，检查系统的网络拓扑结构、容量设计是否合理。
开发阶段测试的主要任务是：代码测试及组件测试，检查设计的代码能否满足规格需求。
运行阶段测试的主要任务是： 功能测试、性能测试、安全性测试、配置测试、兼容性测试及易用性测试。
维护阶段测试的主要任务是：根据维护的内容实施开发及运行阶段中的各个相关方面的测试。
服务器端容量的规划是否合理，系统的安全设计是否合理，数据库设计是否合理，检查客户端设计的功能是否正确合理，检查系统的网络拓扑结构、容量设计是否合理。开发阶段测试的主要任

测试过程　采用敏捷开发方法
包括 功能测试　内容测试和评审　Web 页面测试　导航测试　接口测试　配置测试
安全测试　性能测试

步骤：

需求分析

定义测试策略和方法

确定测试环境的要求

描述测试细节

功能测试：

1. 链接测试（特有）
   - 链接正确
   - 资源存在
   - 都能跳转链接到不存在的页面
2. 表单测试　用户提交信息的正确性和有效性
   - 标识域约定正确标记
   - 用户输入正确性合法性检查
   - 输入错误的异常处理
   - 合适默认项
   - "后退"不破坏表单
3. 数据校验测试　用户输入正确性合法性校验
4. Cookie 测试
5. 数据库测试　完整性　模式正确
   - 数据完整性错误
   - 输出错误

性能测试：

1. 速度测试：　网络连接速度　业务处理速度
2. 负载测试：　最大并发用户数量
3. 压力测试　不断加压　什么条件下不能承受　找出性能崩溃
4. 强度测试　异常情况的抗打击能力　极限状态下性能下降的幅度是否可接受
5. 并发测试　多个用户同时访问
6. 大数据量测试　运行数据量较大　所处数据量较大
   - 实时大数据量测试
   - 极限状态下测试
7. 配置测试：　系统各项资源最优分配原则　系统调优
   测试一组客户端和服务端配置，确保所有用户体验一样
8. 可靠性测试：　检测系统是否稳定

方法：

虚拟用户方法：　模拟真实用户施加工作负载

WUS　强调建立真实的负载　一系列全面测试负载参数和测量指标的集合

安全性测试：　安全防卫措施的有效性

1. 数据加密测试
2. 用户身份验证测试
3. 日志文件测试
4. Session 测试
5. 备份与恢复测试
6. 访问控制策略测试
7. 安全漏洞测试
8. TCP 端口测试
9. 服务器端脚本漏洞检查

7. 基于漏洞测试
8. TCP 端口测试
9. 服务端脚本漏洞检查
10. 防火墙测试

服务测试

特性

表 16-1 Web 服务测试特性

测试因素	Web 服务测试
测试参与者	需要服务的提供者、代理和请求者共同参与到测试过程中
测试分布	分布、远程、多阶段
测试模型	不拥有服务代码（服务开发者除外），有限的测试模型（可控制性和可观测性低）
测试覆盖	除服务开发者拥有传统的覆盖范围外，服务提供者、服务集成者、服务请求者和服务代理可能只有黑盒覆盖范围
测试执行	在线、及时测试
测试客户端	需要构建测试客户端调用被测服务
测试预测	Web 服务动态绑定，很难预测其实际运行情况，难以生成测试预测
回归测试	在线、基于运行时收集数据的测试；除服务的开发者和提供者外；服务请求者、服务集成者和服务代理并不掌握服务的演化情况，回归测试需要额外的信息支持

内容：

测试组织管理
- Web 服务基础设施的验证与确认　Web 中间件稳定性
- Web 服务独立测试　功能/性能与发布的服务描述一致
- Web 服务集成测试　服务描述正确描述业务需求　能被服务中介

多个服务测试比较评估

建立一个能机制支持不同版本的维护和回归测试

嵌入式系统分析与设计

2025年5月21日 星期三 21:47

以应用为中心 以计算机为基础
可裁剪 可裁减的软硬件为一体的专用计算机系统

特点：专用性强 软硬件结合紧密
实时性强 限时响应
软硬件依赖性强 专用性决定的
专用处理器 为特定目的 专门设计的
多种技术紧密结合
系统透明性 形态差异大 用户感知弱
系统资源要求

组成：
传感器 物理参数 → 电信号
处理单元 接收数字信号 并处理
作动器 接收电信号 执行物理动作
输入接口 电信号 → 数字形式
输出接口 二进制 → 电信号

体系架构：
硬件层 嵌入式微处理器 存储器 通用设备接口 I/O 接口
中间层 硬件抽象层 板级支持包 使底层驱动程序与硬件无关
包含底层硬件初始化 数据输入与输出操作 和硬件设备的配置功能
系统软件层 实时多任务操作系统 文件系统 图形用户接口 嵌入式数据库 网络系统 通用组件模块
功能层 实现任务 的程序

硬件 嵌入式处理器 存储器 外部设备
软件 系统软件 应用支撑软件 应用软件

多核处理器 低功耗 多线程并发
1. 单核 只有一个物理处理器 不可并行
2. 多处理器结构 真正并行
3. 超线程结构 SMT 一个实际物理处理器 多个逻辑处理器
4. 多核结构 单芯片多处理器 CMP 设计 单芯片内实现两个或更多的执行核
5. 共享Cache的多核结构 执行核共享片上 Cache
6. 采用超线程技术的多核结构 多核+超线程 逻辑处理器=执行核的2倍

安全攸关系统 严重后果
单位时间内损失的期望值小于规定的概率

安全设计模式：
三模块冗余 表决器 没有 fail-safe 状态
监控-启动器模式 独立传感器监控系统 有 fail-safe 状态

数据库管理系统 对资源要求相对有限 固化在嵌入式计算机
完整的管理系统包含
特点 嵌入性 实时性
连接网络 嵌入式数据库 同步服务器 数据库服务器
独立 一致性

特点：
嵌入性
实时性
移动性
伸缩性

连接网络 { 嵌入式数据库 / 同步服务器 / 数据库服务器 }

分类
一致性

功能划分角度：内存数据库管理系统　离散 简单　内存
实时数据库管理系统　连续 实时　外存
移动数据库管理系统　远程　独立计算

数据库系统
包含管理系统 硬件 软件

使用环境特点：
1、设备随时移动
2、网络频繁断开
3、网络条件多样化
4、通信能力不对称

嵌入式操作系统
负责全部软硬件资源分配，任务调度 控制 协调并行活动
包括 与硬件相关的底层驱动软件，系统内核 设备驱动接口 通信协议 图形界面 标准化 浏览器

响应时间角度：硬实时 软实时

特点：
微型化
代码质量高
专业化
实时性强
可裁减 可配置

结构和组成　层次化架构　隔离用户和计算机系统
由 硬件抽象层 调试代理 操作系统内核 文件系统 可配置组件 组件

硬件抽象层　操作系统建立在 抽象层上　完成以接口

级别：
系统结构抽象层　CPU核 特征
处理器变种抽象层　CPU变种特征
私抽象层　抽象不同私的特征

内核：　任务管理 存储管理 任务间通信 异常和中断管理

包含 调度器　心脏
内核对象　内核构件
内核服务　对象上执行操作或通用操作

分类：
宏内核　用户与内核在同一空间 实时性高 结构简单 易于设计 但代码耦合度高 缺少移植性
微内核　用户与内核在不同空间 结构清晰 利于协作开发 系统性能偏低
混合内核　微内核中运行在用户层的功能模块 放回内核中 不常用 耗时长 还是在用户层

实时性能：由硬件 实时操作系统　应用程序共同决定
RTOS内核起关键作用

指标：
任务切换时间　CPU控制权由运行态的任务转到就绪任务的时间

指标：

任务切换时间　CPU 控制权由运行态的任务转到就绪任务的时间

中断处理相关的时间指标：

中断延迟时间　中断产生到系统感知中断

中断响应时间　中断产生到开始执行中断服务例程

中断恢复时间　中断服务例程结束回到被中断的代码

任务响应时间　任务中断产生到任务真正开始

系统响应时间　系统发出处理请求到系统做出响应的时间

多任务调度算法

软时限与硬时限算法：软　尽快执行

硬　准时完成

周期性与非周期性算法：调度周期性

非周期性 不能预期

可抢占与非抢占算法：可抢占　正在运行的任务可能被打断 放弃CPU

非抢占　任务会一直运行到结束或阻塞

静态算法与动态算法：静态　优先级在设计时确认

动态　运行过程中动态确认优先级

单处理器与多处理器调度算法：单　只处理一个处理器的情况

多

在线与离线调度算法：离线　运行前确定

在线　运行过程中收集信息确定

广泛支持　基于优先级的抢占调度和时间轮转调度

单调速率调度　固定静态优先级算法　单调函数

最频繁最高　周期越短越高

时间轮转调度　运行一个小时间片　再调度另一个

截止时间优先　执行时限越近　优先级越高　动态调整

基于分区化的调度 APPS　多个确定的时间片　每个分区实体使用自己的时间片资源

分区　调度一般使用固定时间优先调度　周期不的分区优先安排

优先级反转　资源占用

时间 未知 无界 / 时间 已知 有界 → 优先级反转

解决：资源控制协议：优先级继承协议　动态改变任务优先级，可能死锁

优先级天花板　直接提到最高优先级　不会死锁

嵌入式系统开发：交叉开发 在通用机开发，在目标机运行

交叉开发环境：交叉开发工具和实用程序

实时操作系统

连接宿主机与目标机的多种通信方式

宿主机提供：交叉编译器　交叉链接器　源代码调试器

目标机提供：动态装载器　链接装载器　监视器　调试代理

开发流程：项目计划　需求分析　概要设计　详细设计　编码实现　下载调试　固化　测试　运行

常用模型：瀑布模型　螺旋模型（瀑布+快速原型

支持迭代模型：快速面向对象过程 RooPES　UML　螺旋式迭代　半螺旋式（瀑布+半螺旋）

无线互联网服务工程过程 WISEP

分析和设计方法：

1. 改进的结构化方法：实时结构化分析与设计　实时系统设计方法　并发实时系统设计方法.
（不利于复用性和可修改性）
2. 面向对象的方法：　实时面向对象建模　实时系统面向对象技术
复用性和可修改性优势
3. 基于构件的开发方法　清晰的接口和良好的复用机制
4. 基于模型的开发方法：　用于初期设计阶段　描述受控对象和控制计算机系统的行为

分析和设计过程

1. 需求分析 → 系统规格说明书　采用UML进行需求描述
2. 系统架构设计　先考虑软件架构　再考虑对硬件实现
包括系统实时性要求　采用的设计模型　是否需要 EOS（嵌入式操作系统）
3. 硬件子系统设计：目标硬件平台　EOS　编程语言　开发工具
4. 软件子系统设计　根据实时性要求　应用场景　可用资源等约束
① 明确实时性指标，软实时 硬实时
② 进行可调度分析
③ 根据系统特点确认优先级
5. EOS选择
6. 编程语言选择

软硬件协同设计　使用统一的方法和工具对软硬件进行描述 综合与验证
设计时充分考虑软硬件资源情况

协同设计工具：协同仿真工具 协同模拟工具

低功耗
- 软件　编译优化技术　软硬件协同设计　算法优化
- 硬件　降低工作电压
  - 极级电路低功耗技术
  - 选择低功耗处理器
  - 总线的低功耗设计
  - 接口驱动电路的设计
  - 分区分时供电技术

嵌入式系统验证

验证：已满足我们的要求　正确地做事

确认：要求得到满足　做正确的事

测试：评审　分析　测试

传统的软件测试：基于过程的测试方法　双V型

问题：测试不够彻底
编码完成后开始测试

测试人员理解不够
文档优劣影响测试效果
缺乏自动化测试
修复一个错误又引入新的问题

基于需求的软件测试方法 DO-178B/C

DO-178B/C 软件生命周期分类
1. 软件计划过程
2. 软件开发过程 包括 软件需求过程 设计过程 编码过程 集成过程
3. 软件综合过程 包括 软件验证过程 软件配置管理过程 软件质量保证过程 审定联络过程

测试方法：软硬件集成测试
软件集成测试
低级需求的测试

基于需求的覆盖率分析 RCA 对软件需求的覆盖程度
基于结构的覆盖率分析 SCA 对代码结构的覆盖程度
代码未覆盖处理原则

有需求 无用例	加用例
无需求	加需求加用例
死代码	提出问题报告删除
未激活代码	分析

形式化方法：基于数学方法 采描述 目标软件系统属性的技术
用于发现其他方法难以发现的系统描述不一致 不明确 不完整的情况

优点：
面向对象
面向属性
基于并发性
基于实时性
基于系统的构件测试方法：

过程：单元测试 发现构件本身的错误
任务：白盒测试 确认构件的内部逻辑 数据和程序结构
黑盒测试 确认构件的外部功能 行为和接口
性能
集成测试：构件和应用软件交互的错误
任务：通信和交互关系
构件的集成顺序和如何测试新集成的构件
构件是否符合系统需求
系统测试：测试整个系统的功能与性能
回归测试：
任务：确定变化
分析变化影响
选择 可复用的测试用例
增加新的测试用例

微服务系统分析与设计

2025年5月23日 星期五 01:33

开发软件的架构和组织方法　小型自治的服务　每个服务独立部署 运行和维护　通过轻量级通信机制相互协作

优势：
独立性和自治性
弹性和可伸缩性
技术多样性
专用性
可组合性和可扩展性
容错性和可恢复性

实现方式
基于容器化技术
RESTful API　HTTP HTTPS
消息队列　异步
服务注册和发现　动态发现和调用
服务网关　入口和流量路由器　提供均衡负载 鉴权认证 请求转发 缓存等功能
分布式数据库

特征
服务自治性　独立代码库 数据库 和团队　独立开发 部署和运行
服务单一职责　专注于解决一个特定的业务问题　内聚性 可维护性
服务松耦合　独立演化和变更
分布式部署　部署在不同的服务器或容器中　高并发 大数模的需求
技术异构性　使用不同的技术栈和工具　灵活选择和使用最适合的技术
弹性和可伸缩性　自动调整
独立演化和部署　不影响其他服务

主要原则：
单一职责原则
隔离性原则
自治性原则
弹性原则
可恢复性原则
可测试性原则
可维护性原则
安全性原则
智能端点和简单消息传递

架构模式
聚合器微服务　多个服务或数据源中 收集 处理 聚合 信息
聚合器是个中心化服务 接收客户端请求 并发给后端

具备功能：
1. 消息转发　转发给后端服务
2. 数据聚合　数据聚合转换 生成所需的数据格式

聚合功能：1. 消息转发　转发给后端服务
2. 数据聚合　数据聚合转换　生成所需的数据格式
3. 数据缓存　缓存聚合后的数据　避免重复处理

优点：减少网络延迟　减少微服务之间的依赖　提高系统可靠性　提高开发效率

代理微服务　仅负责请求　或进行数据转换工作　不聚合数据
接收和转发客户端请求到后端服务的中间层
可以实现常见服务治理功能：负载均衡　故障熔断　限流　服务发现

组成部分　代理网关　后端服务

HTTP 或 TCP

链式微服务：多个微服务连接在一起　形成一条微服务链　处理结果传递给下一个微服务

分支微服务：链式微服务的扩展
多个微服务被组织到一起　形成不同的分支结构　每个分支代表一种可能的业务场景或决策路径
根据输入和条件确定果取的下一步行动

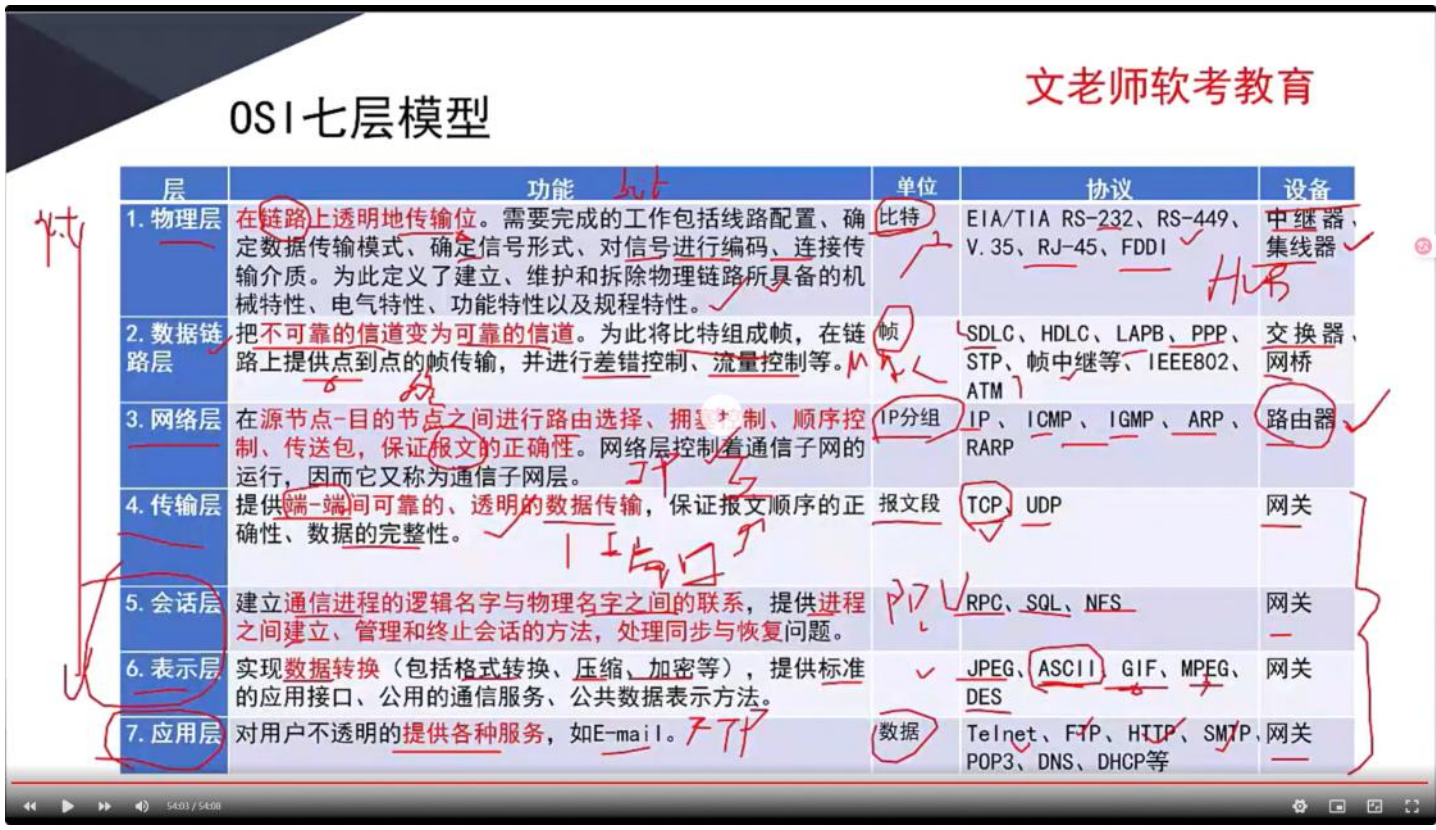
数据共享微服务　在不同微服务之间进行数据共享
将数据访问和管理集中在一个或多个微服务上

方法　1. 共享数据库
2. 数据代理服务

异步消息微服务：基于消息传递的微服务架构　异步通信　无需等待响应返回　立即执行其他任务
每个微服务充当消息的生产者和消费者
消息队列　事件驱动架构　高度松耦合的系统设计
高性能　可伸缩　可靠的分布式系统

OSI七层模型

2025年4月28日 21:45



TCP/IP

2025年4月29日 星期二 22:47

网络协议 三要素：语法、语义 时序

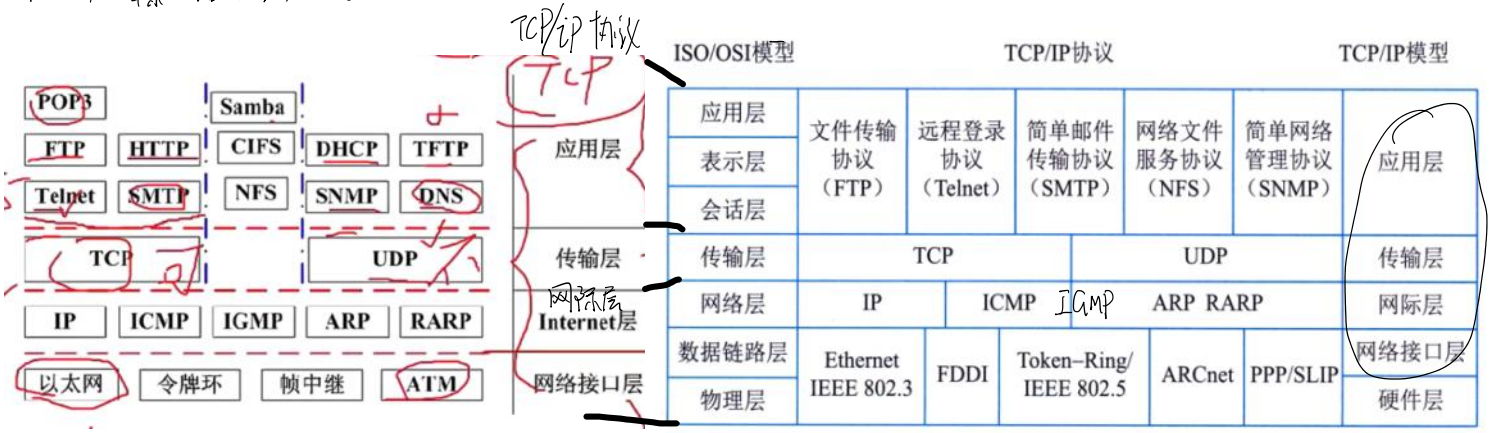


图 4-4 TCP/IP 模型与 OSI 模型的对比

IP数据报 长度 20～60

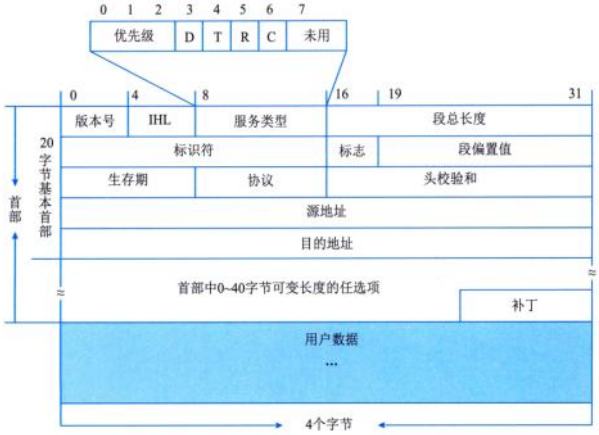


图 4-6 IP 协议格式

TCP协议

源端口								目标端口
发送顺序号								
接收顺序号								
偏置值	保留	URG	ACK	PSH	RST	SYN	FIN	窗口
校验和								紧急指针
任选项+补丁								
用户数据								

图 4-8 TCP 传输头格式

IP地址

32位 点分十进制

ABC类 无分类编址 (子网 网络号 主机号)

静态IP地址

公有

私有

公有
私有

169.254 DHCP

子网划分

计算

子网掩码 主机号为0

聚合超网

IPv6

128位

灵活IP报头

过渡

双协议栈

隧道技术

翻译技术

数字数据编码

2025年4月25日 14:33

1)不归零编码(Non-Return-Zero,NRZ)
用低电平表示二进制0,用高电平表示二进制1
缺点是无法判断每一位的开始与结束，收发双方不能保持同步。为保证收发双方同步，必须在发送不归零编码的同时用另一个信道传送同步信号。
2)曼彻斯特编码(Manchester Encoding,ME)
不用电平的高低表示二进制，而是用电平的跳变来表示的。在曼彻斯特编码中，每一个比特的中间均有一个跳变，这个跳变既作为时钟信号，又作为数据信号。
电平从高到低的跳变表示二进制1,从低到高的跳变表示二进制0
3)差分曼彻斯特编码(Differential Manchester Encoding, DME)对曼彻斯特编码的改进
每比特中间的跳变仅做同步之用，每比特的值根据其开始边界是否发生跳变来决定。每比特的开始无跳变表示二进制1,有跳变表示二进制0
曼彻斯特编码和差分曼彻斯特编码是数据通信中最常用的数字信号编码方式，它们的优点是无须另发同步信号；缺点是编码效率低，如果传送10Mb/s的数据，将需要20MHz的脉冲。

4)多电平编码
这种编码的码元可取多个电平之一，每个码元可代表几个二进制位。例如，令M=2”,设M=4,则n=2。若表示码元的脉冲取4个电平之一，则一个码元可表示两个二进制位。与双相码相反，多电平码的数据速率大于波特率，因而可提高频带的利用率。但是这种代码的抗噪声特性不好，在传输过程中信号容易畸变到无法区分。
5)4B/5B编码
在曼彻斯特编码和差分曼彻斯特编码中，每位中间都有一次电平跳变，因此波特率是数据速率的两倍。对于100Mb/s的高速网络，如果采用这类编码方法，就需要200MBaud(波特率),其硬件成本是100MBaud硬件成本的5～10倍。为了提高编码的效率，降低电路成本，可以采用4B/5B编码，即用5bit的二进制码来代表4bit的二进制码。

结构化布线系统

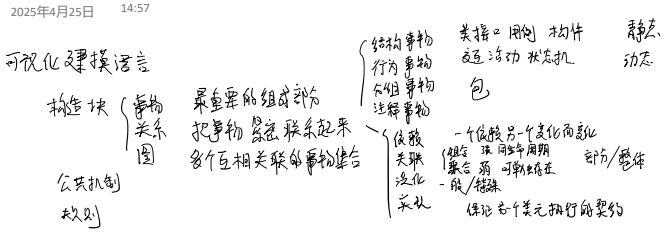
2025年4月25日 14:43

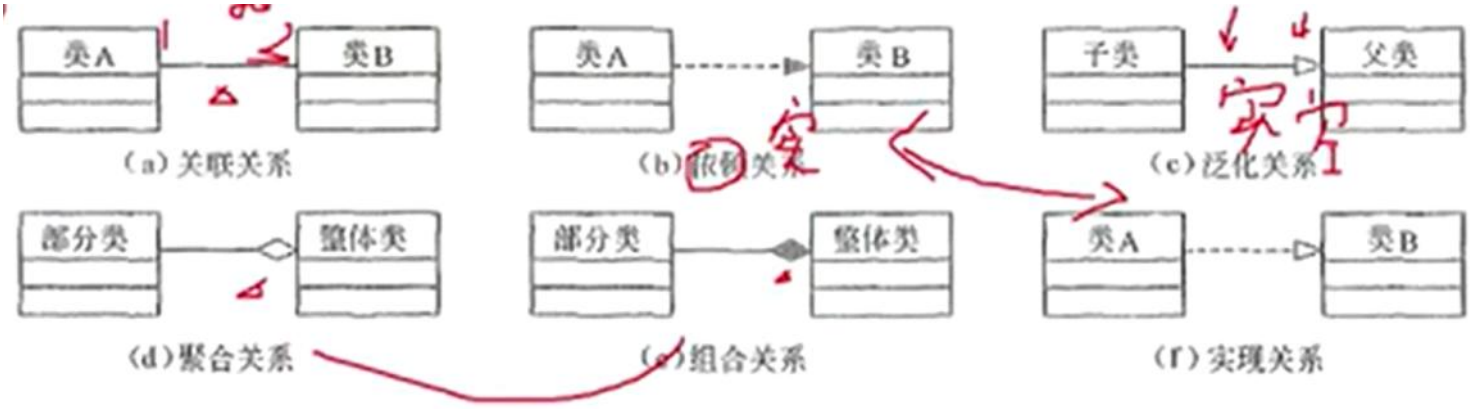
- 1. 工作区子系统：**信息插座（或通信接口）到终端设备**
- 2. 水平子系统：设备间或管理间延伸到工作区的信息插座
- 3. 干线子系统（垂直子系统）：连接建筑物内的不同楼层或建筑群的设备间、管理间等重要网络节点
- 4. 设备间子系统：放置网络硬件设备（如服务器、交换机、路由器等）的地方，所有干线和水平子系统连接的枢纽，布线系统的核心部分
- 5. 管理子系统：布线系统中的配线部分，配线架、跳线及相关的标识系统，进行楼层内配线架之间的交叉连接
- 6. 建筑群子系统：连接园区内各建筑物之间的网络设备

参考文档：详解结构化综合布线系统（包含园区子系统） 结构化布线系统-CSDN博客

统一建模语言UML

2025年4月25日 14:57





1.UML的结构

(1)构造块。UML有三种基本的构造块，分别是事物(thing)、关系(relationship)和图(diagram)。事物是UML的重要组成部分，关系把事物紧密联系在一起，图是多个相互关联的事物的集合。

(2)公共机制。公共机制是指达到特定目标的公共UML方法，主要包括规格说明(详细说明)、修饰、公共分类(通用划分)和扩展机制4种。规格说明是事物语义的细节描述，它是模型真正的核心；UML为每个事物设置了一个简单的记号，还可以通过修饰来表达更多的信息；UML包括两组公共分类，分别是类与对象(类表示概念，而对象表示具体的实体)、接口与实现(接口用来定义契约，而实现就是具体的内容);扩展机制包括约束(扩展了UML构造块的语义，允许增加新的规则或修改现有的规则)、构造型(扩展UML的词汇，用于定义新的构造块)和标记值(扩展了UML构造块的特性，允许创建新的特殊信息来扩展事物的规格说明)。

(3)规则。规则是构造块如何放在一起的规定，具体包括：为构造块命名；赋予这些名称特定含义的上下文，即范围；怎样使用或看见名字，即可见性；事物如何正确、一致地相互联系，即完整性；运行或模拟动态模型的含义是什么,即执行。

UML对系统架构的定义是系统的组织结构，包括系统分解的组成部分，以及它们的关联性、交互机制和指导原则等提供系统设计的信息。具体来说，就是指以下5个系统视图：

(1)**逻辑视图**。逻辑视图也称为**设计视图**，它表示了设计模型中在架构方面具有重要意义的部分，即**类、子系统、包和用例实现的子集。**

(2)**进程视图**。进程视图是**可执行线程和进程作为活动类的建模**，它是**逻辑视图的一次执行实例，描述了并发与同步结构。**

(3)**实现视图**。实现视图对组成基于系统的**物理代码的文件和构件进行建模。**（开发视图）

(4)**部署视图**。部署视图把**构件部署到一组物理节点上**，表示软件到硬件的映射和分布结构。（物理视图）

(5)**用例视图**。用例视图是**最基本的需求分析模型。**（场景）

另外，UML还允许在一定的阶段隐藏模型的某些元素、遗漏某些元素，以及不保证模型的完整性，但模型要逐步地达到完整和一致。

2.事物

UML中的事物也称为建模元素，包括结构事物(Structural Things)、行为事物(Behavioral Things,动作事物)、分组事物(Grouping Things)和注释事物(Annotational Things,注解事物)。这些事物是UML 模型中最基本的 00构造块。

(1)结构事物。结构事物在模型中属于最静态的部分，代表概念上或物理上的元素。UML有7种结构事物，分别是类、接口、协作、用例、活动类、构件和节点。类是描述具有相同属性、方法、关系和语义的对象的集合，一个类实现一个或多个接口；接口是指类或构件提供特定服务的一组操作的集合，接口描述了类或构件的对外的可见的动作；协作定义了交互的操作，是一些角色和其他事物一起工作，提供一些合作的动作，这些动作比事物的总和要大；用例是描述一系列的动作，产生有价值的结果，在模型中用例通常用来组织行为事物，用例是通过协作来实现的；活动类的对象有一个或多个进程或线程，活动类和类很相似，只是它的对象代表的事物的行为和其他事物是同时存在的；构件是物理上或可替换的系统部分，它实现了一个接口集合；节点是一个物理元素，它在运行时存在，代表一个可计算的资源，通常占用一些内存且具有处理能力，一个构件集合一般来说位于一个节点，但有可能从一个节点转到另一个节点。

(2)行为事物。行为事物是UML模型中的动态部分，代表时间和空间上的动作。UML有两种主要的行为事物。第一种是交互(内部活动),交互是由一组对象之间在特定上下文中，为达到特定目的而进行的一系列消息交换而组成的动作。交互中组成动作的对象的每个操作都要详细列出，包括消息、动作次序(消息产生的动作)、连接(对象之间的连接)。第二种是状态机，状态机由一系列对象的状态组成。

(3)分组事物。分组事物是UML模型中组织的部分，可以把它们看成是个盒子，模型可以在其中进行分解。UML只有一种分组事物，称为包。包是一种将有组织的元素分组的机制。与构件不同的是，包纯粹是一种概念上的事物，只存在于开发阶段，而构件可以存在于系统运行阶段。

(4)注释事物。注释事物是UML模型的解释部分。

3.关系

UML用关系把事物结合在一起，主要有下列4种关系：

(1)依赖(dependency)。依赖是两个事物之间的语义关系，其中一个事物发生变化会影响另一个事物的语义。

(2)关联(association)。关联描述一组对象之间连接的结构关系。

(3)泛化(generalization)。泛化是一般化和特殊化的关系，描述特殊元素的对象可替换一般元素的对象。

(4)实现(realization)。实现是类之间的语义关系，其中的一个类指定了由另一个类保证执行的契约。

4.图

UML2.0包括14种图，分别列举如下：结构图 行为图

(1)类图(Class Diagram)。类图描述一组类、接口、协作和它们之间的关系。在00系统的建模中，最常见的图就是类图。类图给出了系统的静态设计视图，活动类的类图给出了系统的静态进程视图。

(2)对象图(Object Diagram)。对象图描述一组对象及它们之间的关系。对象图描述了在类图中所建立的事物实例的静态快照。和类图一样，这些图给出系统的静态设计视图或静态进程视图，但它们是从真实案例或原型案例的角度建立的。

(3)构件图(Component Diagram)，又叫组件图。构件图描述一个封装的类和它的接口、端口，以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。对于由小的部件构建大的系统来说，构件图是很重要的。构件图是类图的变体。展现一组构件之间的组织和依赖

(4)组合结构图(Composite Structure Diagram)。组合结构图描述结构化类(例如，构件或类)的内部结构，包括结构化类与系统其余部分的交互点。组合结构图用于画出结构化类的内部内容。

(5)用例图(Use Case Diagram)。用例图描述一组用例、参与者及它们之间的关系。用例图给出系统的静态用例视图。这些图在对系统的行为进行组织和建模时是非常重要的。用例之间的关系有扩展、包含、泛化。

(6)顺序图(Sequence Diagram,也称为序列图)。顺序图是一种交互图(Interaction Diagram),交互图展现了一种交互，它由一组对象或参与者以及它们之间可能发送的消息构成。交互图专注于系统的动态视图。顺序图是强调消息的时间次序的交互图。

(7)通信图(Communication Diagram)。通信图也是一种交互图，它强调收发消息的对象或参与者的组织结构。顺序图和通信图表达了类似的基本概念，但它们所强调的概念不同，顺序图强调的是时序，通信图强调的是对象之间的组织结构(关系)。在UML 1.X版本中，通信图称为协作图(Collaboration Diagram)。

(8)定时图(Timing Diagram,计时图)。定时图也是一种交互图，它强调消息跨越不同对象或参与者的实际时间，而不仅仅关心消息的相对顺序。

(9)状态图(State Diagram)。状态图描述一个状态机，它由状态、转移、事件和活动组成。状态图给出了对象的动态视图。它对于接口、类或协作的行为建模尤为重要，而且它强调事件导致的对象行为，这非常有助于对反应式系统建模。事件触发器、状态转换

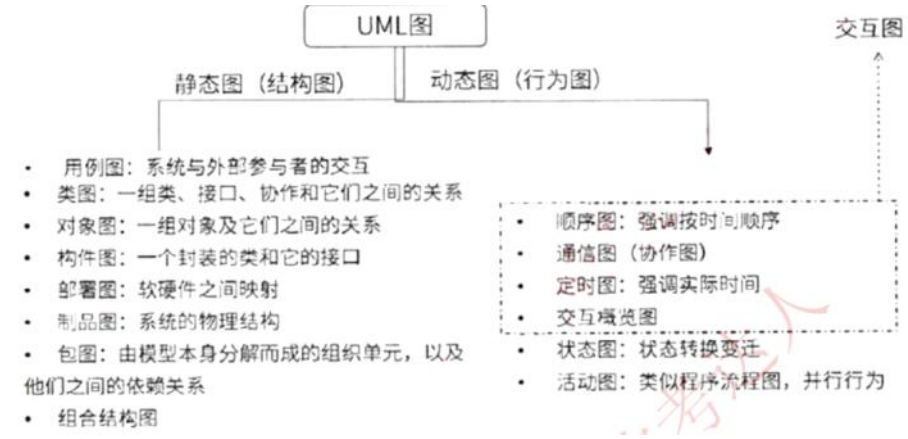
(10)活动图(Activity Diagram)。活动图将进程或其他计算结构展示为计算内部的控制流和数据流。活动图专注于系统的动态视图。它对系统的功能建模和业务流程建模特别重要，并强调对象间的控制流程。特殊的状态图 在系统中从一个活动到另一个活动的流程 并发分叉、并发汇合、监护表达式、分支、流

(11)部署图(Deployment Diagram)。部署图描述对运行时的处理节点及在其中生存的构件的配置。部署图给出了架构的静态部署视图，通常一个节点包含一个或多个部署图。物理模块

(12)制品图(Artifact Diagram)。制品图描述计算机中一个系统的物理结构。制品包括文件、数据库和类似的物理比特集合。制品图通常与部署图一起使用。制品也给出了它们实现的类和构件。

(13)包图(Package Diagram)。包图描述由模型本身分解而成的组织单元，以及它们之间的依赖关系。

(14)交互概览图(Interaction Overview Diagram)。交互概览图是活动图和顺序图的混合物。



用例关系

2025年4月25日 15:14

5.调整用例模型

在建立了初步的用例模型后，还可以利用用例之间的关系来调整用例模型。用例之间的关系主要有包含、扩展和泛化，利用这些关系，把一些公共的信息抽取出来，以便于复用，使得用例模型更易于维护。

(1)包含关系。当可以从两个或两个以上的用例中提取公共行为时，应该使用包含关系来表示它们。其中这个提取出来的公共用例称为抽象用例，而把原始用例称为基本用例或基础用例。

(2)扩展关系。如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，这样使描述可能更加清晰。

(3)泛化关系。当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。在用例的泛化关系中，子用例是父用例的一种特殊形式，子用例继承了父用例所有的结构、行为和关系。

在实际工作中，系统分析师要谨慎选用这些关系。从上面的介绍可以看出，包含、扩展和泛化关系都会增加用例的个数，从而增加用例模型的复杂度。另外，一般都是在用例模型完成之后才对它进行调整，在用例模型建立之初不必急于抽象用例之间的关系。

调整用例模型

2025年4月25日 15:36

依赖关系{(1)包含关系。当可以从两个或两个以上的用例中提取公共行为时，应该使用包含关系来表示它们。其中这个提取出来的公共用例称为抽象用例，而把原始用例称为基本用例或基础用例
(2)扩展关系。如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，这样使描述可能更加清晰
(3)泛化关系。当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。在用例的泛化关系中，子用例是父用例的一种特殊形式，子用例继承了父用例所有的结构、行为和关系。

敏捷开发

2025年10月31日 15:01

① XP。XP（极限编程）的思想源自 Kent Beck和Ward Cunningham在软件项目中的合作经历。XP注重的核心是沟通、简明、反馈和勇气。因为知道计划永远赶不上变化，XP无需开发人员在软件开始初期做出很多的文档。XP提倡测试先行，为了将以后出现bug的几率降到最低。敏捷开发的典型方法之一，是一种轻量级（敏捷）、高效，低风险、柔性、可预测的、科学的软件开发方法，它由价值观、原则、实践和行为4个部分组成。其中4大价值观为沟通、简单性、反馈和勇气。

② SCRUM。SCRUM是一种迭代的增量化过程，用于产品开发或工作管理。它是一种可以集合各种开发实践的经验化过程框架。SCRUM中发布产品的重要性高于一切。 该方法由Ken Schwaber 和 Jeff Sutherland 提出，旨在寻求充分发挥面向对象和构件技术的开发方法，是对迭代式面向对象方法的改进。用迭代的方法，其中把每30天一次的迭代称为一个“冲刺”，并按需求的优先级来实现产品。多个自组织和自治小组并行地递增实现产品。协调是通过简短的日常会议来进行的。

③ Crystal Methods。Crystal Methods（水晶方法族）由Alistair Cockburn在20世纪90年代末提出。之所以是个系列，是因为他相信不同类型的项目需要不同的方法。虽然水晶系列不如XP的产出效率高，但会有更多的人能够接受并遵循它。水晶方法体系与XP一样，都有以人为中心的理念，但在实践上有所不同。水晶方法体系考虑到人们一般很难严格遵循一个纪律约束很强的过程，认为每一种不同的项目都需要一套不同的策略、约定和方法论。因此，与XP的高度纪律性不同，水晶方法体系探索了用最少纪律约束而仍能成功的方法，从而在产出效率与易于运作上达到一种平衡。也就是说，虽然水晶系列不如XP那样的产出效率，但会有更多的人能够接受并遵循它。

④ FDD。FDD（特性驱动开发）由Peter Coad、Jeff de Luca 、Eric Lefebvre共同开发，是一套针对中小型软件开发项目的开发模式。此外，FDD是一个模型驱动的快速迭代开发过程，它强调的是简化、实用、易于被开发团队接受，适用于需求经常变动的项目。

⑤ ASD。ASD（自适应软件开发）由Jim Highsmith在1999年正式提出。ASD强调开发方法的适应性，这一思想来源于复杂系统的混沌理论。ASD不像其他方法那样有很多具体的实践做法，它更侧重为ASD的重要性提供最根本的基础，并从更高的组织和管理层次来阐述开发方法为什么要具备适应性。ASD的核心是三个非线性的、重迭的开发阶段：猜测，合作与学习

⑥ DSDM。DSDM（动态系统开发方法）是众多敏捷开发方法中的一种，它倡导以业务为核心，快速而有效地进行系统开发。实践证明DSDM是成功的敏捷开发方法之一。在英国，由于其在各种规模的软件组织中的成功，它已成为应用最为广泛的快速应用开发方法。DSDM不但遵循了敏捷方法的原理，而且也适合那些成熟的传统开发方法有坚实基础的软件组织。

⑦ 轻量型RUP。RUP其实是个过程的框架，它可以包容许多不同类型的过程， Craig Larman极力主张以敏捷型方式来使用RUP。他的观点是：目前如此众多的努力以推进敏捷型方法，只不过是在接受能被视为RUP 的主流OO开发方法而已。

来自 <https://ruankaodaren.com/exam/#/mockTestResult/mockTestResult?examId=1984086039389671424&type=8>

国密算法

2025年11月1日 15:28

SM1：对称加密，分组长度和密钥长度都为128比特。广泛应用于电子政务、电子商务及国民经济的各个应用领域

SM2：非对称加密，用于公钥加密算法、密钥交换协议、数字签名算法。国家标准推荐使用素数域256位椭圆曲线

SM3：杂凑算法，杂凑值长度为256比特。适用于商用密码应用中的数字签名和验证

SM4：对称加密，分组长度和密钥长度都为128比特。适用于无线局域网产品

SM9：标识密码算法。不需要申请数字证书，适用于互联网应用的各种新兴应用的安全保障

来自 <https://ruankaodaren.com/exam/#/mockTestResult/mockTestResult?examId=1984511620273147904&type=8>

软件模块之间的耦合度

2025年11月1日 15:34

软件模块之间的耦合度从低到高排序为：

1. 非直接耦合：两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的。
2. 数据耦合：一组模块借助参数表传递简单数据。
3. 标记耦合：一组模块通过参数表传递记录信息（数据结构）。
4. 控制耦合：模块之间传递的信息中包含用于控制模块内部逻辑的信息。
5. 外部耦合：一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息。
6. 公共耦合：多个模块都访问同一个公共数据环境，公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等。
7. 内容耦合：一个模块直接访问另一个模块的内部数据；一个模块不通过正常入口转到另一个模块的内部；两个模块有一部分程序代码重叠；一个模块有多个入口。

来自 <https://ruankaodaren.com/exam/#/mockTestResult/mockTestResult?examId=1984511620273147904&type=8>

IDEF

2025年11月5日 11:04

IDEF是一系列建模、分析和仿真方法的统称，从IDEF0到IDEF14（包括IDEF1X在内）共有16套方法，每套方法都是通过建模程序来获取某个特定类型的信息。它们分别是IDEF0（功能建模）、IDEF1（信息建模）、IDEF1X（数据建模）、IDEF2（仿真建模设计）、IDEF3（过程描述获取）、IDEF4（面向对象设计）、IDEF5（本体论描述获取）、IDEF6（设计原理获取）、IDEF7（信息系统审计）、IDEF8（用户界面建模）、IDEF9（场景驱动信息系统设计）、IDEF10（实施架构建模）、IDEF11（信息制品建模）、IDEF12（组织建模）、IDEF13（三模式映射设计）和IDEF14（网络规划）。

IDEF0的建模特点使它可以用来描述企业的业务流程，它的阶梯层次可用来描述业务流程的阶梯结构特性。从高层次看，IDEF0的功能活动与业务流程相对应；而从低层次看，功能活动与流程的业务活动相对应。利用IEDF0的活动描述方式及活动之间的联系方式，可以很好地描述业务流程的架构。IDEF0模型形象、直观、易于理解和分析，但是，这种图形化的模型没有深刻揭示业务流程的内部结构特征和规律，而且当业务流程很复杂时，所对应的有向图就成为一个相互交叉、混乱的网络，不利于分析流程的特征。

来自 <https://ruankaodaren.com/exam/#/mockTestResult/mockTestResult?examId=1985898543484899328&type=8>

考情

2025年4月29日 9:50

主题	系分论文考查情况
系统规划与战略规划	0-1道
需求工程	0-2道
系统开发方法	0-2道
系统维护和系统测试	0-2道
项目管理	0-1道
面向对象技术	0-1道
软件过程改进实践	较少考察
应用系统分析与设计	0-1道
数据库工程	0-1道
系统安全	较少考察
应用系统集成	0-1道
新技术的应用	0-2道

来自 <https://zhuanlan.zhihu.com/p/26981294158>

考期	考题
2024年下半年	论devops在企业信息系统开发中的应用 论系统业务流程分析方法及应用 论软件测试方法及应用 论信息系统运维管理
2024年上半年	论基于架构的软件设计方法 论性能测试方法及其应用 论云原生应用开发 论多源数据集成方法及其应用
2023年上半年	论信息系统可行性分析 论Devops及其应用 论敏捷开发方法 论信息系统数据转换和迁移
2022年上半年	论原型法及其在信息系统开发中的应用 论面向对象涉及方法及其应用
2021年上半年	论面向对象分析方法 论静态测试方法及应用 论RIA富客户端框架应用 论DevSecOpa技术应用
2020年下半年	论软件设计模式及应用 论快速应用开发（RAD） 论面向服务的开发方法 论遗产系统演化

来自 <https://zhuanlan.zhihu.com/p/1897669266259551536>

需要掌握的知识

2025年5月14日 9:58

软件工程 开发模型 开发方法	2500字	排除的论题
系统分析 各种方法	摘要300字	数据仓库
系统设计 结构化 面向对象		
系统测试 黑盒白盒灰盒		
需求工程 结构化需求分析 面向对象UML		
网络与信息安全		

(1)简要叙述你参与维护的软件的概要和你所担任的工作。
(2)具体叙述在XX过程中,为了XX,你采用过哪些技术、哪些方法和哪些工具?遇到过什么问题?为了解决这些问题,采取过哪些措施?
(3)简要叙述你所采取的措施的效果如何?你现在认为还有哪些需要改进的地方,以及如何改进?

摘要

2025年5月14日 14:31

2024年3月，我作为系统分析师参与了《省级iTV流媒体平台》项目的开发与建设，该项目为某省电信运营商提供4K超清点播与实时直播服务。本文重点论述（方法论名称，如：领域驱动设计/微服务治理框架）在复杂系统构建中的实践路径。通过（需求采集方式，如：深度用户画像分析/多维度业务建模）精准定义系统边界，运用（核心工具，如：结构化分析矩阵/动态领域原型）实现业务逻辑与技术架构的闭环映射。在关键方案设计中，通过（抽象设计策略，如：服务粒度解耦/异常熔断机制）平衡系统扩展性与稳定性，采用（标准化手段，如：契约测试驱动/灰度发布策略）保障迭代质量。项目历时8个月顺利交付，投入运营使用半年来运行状态良好，支撑日均百万级活跃用户的业务规模体量，获得省级数字化创新案例奖，为同类型系统建设提供了可复用的技术范式。

项目背景

2025年5月14日 16:21

(1)简要叙述你参与维护的软件的概要和你所担任的工作。

项目背景（248字）

随着超高清视频产业的快速发展和"数字中国"战略的持续推进，传统视频服务平台普遍面临资源分散、卡顿率高、扩展性不足等痛点。2024年3月，受某省电信运营商委托，我司启动《省级iTV流媒体平台》开发建设项目，旨在构建全省统一的智能视频服务中枢，支撑百万级用户并发的超高清内容传输需求。作为项目的系统分析师，我主导完成跨部门需求协调与架构蓝图设计，重点解决多业务系统数据孤岛破除、异构终端自适应分发等技术治理难题。通过对接省视频云基础设施与三大运营商CDN节点，推动项目纳入"国家文化数字化战略省级示范工程"，获得财政专项资金1200万元及政策资源倾斜，为后续打造区域性视频服务标杆奠定基础。

二、项目整体介绍（253字）

作为系统分析师，我负责统筹技术方案设计与多方资源协调，确保业务需求与技术架构的深度融合。该项目针对省级视频服务领域长期存在的三大痛点：一是资源分散化（各地市自建CDN节点导致内容重复存储率达47%），二是服务低效化（传统单集群架构下高峰时段卡顿率超15%），三是终端碎片化（需适配12类硬件设备及7种协议标准）。平台采用"智能调度中枢+边缘节点自治"的混合架构，重点突破跨域资源协同调度（构建虚拟化内容池减少46%存储冗余）、高并发流媒体传输（动态码率算法降低32%带宽损耗）及多终端自适应渲染（统一编解码引擎提升3倍适配效率）三大技术瓶颈。

主体部分

2025年5月20日 15:58

(2)过渡内容
非核心论点问题的回应
引出主体内容(核心论点) 论点是XX重要性关键.....因此需要重视XXX，XXX主要内容有 包括..... 具体
字数：300~400字
(3)主体内容
采用总分式描述：
“一总”加“三分”“一对三”模式
可分4个段落
字数：1000~1500字

来自 <https://zhuanlan.zhihu.com/p/1897669266259551536>

结尾 反思与总结

2025年5月15日 14:36

(4)论文结论
结构上可分三步走：
先分析项目运行效果
再总结项目不足
最后提解决思路
字数：400~600字

来自 <https://zhuanlan.zhihu.com/p/1897669266259551536>

项目历时8个月交付，上线后整合全省9地市21个独立视频系统，承载（业务规模，如：年均4.8万场次直播/日活用户峰值210万），较旧系统节省32%带宽成本，获评省级数字化创新案例奖，形成可复制的省级视频云建设范式。平台上线后累计服务用户超900万，日均处理4K视频流12PB，成为全国首个通过广电总局超高清平台三级等保认证的省级项目，相关经验已复制至3个兄弟省份推广借鉴。

在项目结束后的讨论会上，大家也指出了项目上存在的一些不足之处。

通过此次项目，我们深刻地认识到……（不足改进），避免出现类似的问题

微服务

2025年5月20日 17:00

论 微服务架构及其应用
近年来，随着互联网行业的 迅猛发，展公司或 组织业务的不断扩 ，张需求的 快速变化以及用户量的不断增加，传统的 单块(Monolithic) 软件架构面临着越 来越多的挑战，已逐渐无法适应互联网时代对软件的要求。在这一 背景下，微服务架构模式 (Microservice Architecture Pattern) 逐渐流行，它强调将单一业务功能 开发 成微服务的形式 ,每 个 微服务运行 在 一个进程中; 采用 HTTP 等通用协议和轻 量 级 API 实现微服务之间的协作与通信。这些微服务可以 使用不同的开发语言以 及不同 数据存储 技术，能够 通过自动化部 署工具独立 发，布并保持最低限制 的集中式管理。

请 围绕“微服务架构及其应用 ”论题，依次从以下三个方面 进行论述。

1、概要叙述你参与 管理和开发的 、采用微服务架构的软件开发 项目及在 其中所担任的主要工作。

2、与单 块架构相比较，微服务 架构有哪些特点？请列举至少 4 个特点并进行 说明。

3、结合你参与 管理和开发 的软件开发 项，目描述该软件的架构，说明该 架构是如何采用微服 务架构模

式 的，并说明在采用微服务架 构 ，后在软件开发过 程中遇到的实际问题和解决 方案。

范例:

摘要部分:

“* 年**月，我单位联合沁 单位 研发了xx 系统。系统 以 “功能为核心，主要分为等 模块，支持** 平台。我 作为 该项目的技术负责人参与 整个开发 过 程，主要负责**。
本 文以该 项目为例，主要论述了微服务架构在 该 系统中的具体 应用。系统划分为 前端Web 服务、平台保障服务、业务服务。前 端 Web 服务由Nginx 负载均衡 与服务器集群 结合，解 决前台界面的并发问题; F台保障服务以Eureka 为中心，分为API 网关 、服务注册中心、监控 平，台用以实现基础服务 框架; 业务服SBT Spring Cloud 开发，分为多个微服务，实现具体业务功能，解 决协同问题。最终系统顺利 上 线，受到用户一致 好评。
【注意: 实际写作中相关项目情况应介绍清楚，摘要字数 (包括 标点符号) 一 般写到290-300 F]
正文部分:

我 在一 个** 的 单位任职，过往成果有** 等。“年 入月，我单位联合党 研发了”项目（以下简称为““*系

统 ”），以取代原有传统 的所 平台。系统 以 “功能为核心，主要分 为“* 等模块。其中** 模 块“。在这个项目中，我作为 该项目的 技术负责人参与整个开发 过，程主要负责**。【第 1 段对系统 整体情况 进行细致介绍，

项目背 景段内容可以写到400 到 450 F]
我 们经 过分析，决 定使用微服务架构来开发*“* 系 统项目。在单 块架构设计中，所有功能模块 都属于一 个庞大的系统，各模块 之间 耦合性太强，无法进行分布式部 ，署单个功能模块也很 难 在 水 平方向上灵活的按需扩展。与单 块架构相比，微服务架构的 优势有: 1. 解决了复杂性问题，应用被分解 为多个服务，每个服务都有消息驱动API 定义清楚的 边界; 2. 提供了模块化的解决 方案，单个服务模块很 容易开发、理解和维护，无需 协调其他服务对本服务的影响; 3. 技 术自，由每个服务都可由专门的开发 团队来开发，可分别 选择合适的技术，只需提供标准的API 服务; 4. 性能与可扩展性，强不同类型 的服务可针对性的部 署在适合资源需求的硬件上，支持弹性配置。
“系统的微服务架构基于Spring Cloud 框架，使用Java 语言 开发，将平台服务划分为三类, 分别为前

端 Web 服务、平台保障服务、业务服务。下面针对这三类服务展开具体 说明。

一、前端Web 服务
前端Web 服务主要提供给用户 使用的 界面，分为前置Nginx 负载均衡服务器、前端网站Nginx 集群。
当用户 通过网络 访问系统 时，首先 会访问到前 置的 Nginx 负载均衡服务 器，负载均衡服务器会将请求转发到

前端网站的 Nginx 集群,前 端网站通过发起Http 请求来和后端交互,具体是通过 Ajax 方式 来调用后端REST

API 接口。用户访问网站通过前 置的 Nginx 负载 均衡服务 器来 转发到前 端网站集群，以 起到将用户 请求进行

分流的作用。当前端网站集群中的部分服务发生故障 时，系统仍可正常地对外提供服务。前置Nginx 负载均衡服务器 使用软件反向代理的方式 来 实现负载 均衡，部署为路由模式，系统内部网络 与外部网 络 分属于不同的 逻辑网 络 ，以实现系统内部 与外部网络的隔离。在负载均衡算法的 选择上， 使用最小连接法，每当用户的请求来临时，任务分发单元 会将任务平滑分配给最小 连接数 的前 端网站节点，这样的架构以 廉价且透了明的方
式扩展 了服务器和网络的带宽，可以大大提升 系统的并发 ，量同时保证网站前 端整体的稳定性和可靠性。

二、平台保障服务

平台保障服务用以实现后 端微服 务的基础 框架, 包括 API 路由网关 、服务注册中心、服务监 控组件 。AP1

论题:论微服务系统开发应用
近年来,随着互联网行业的迅猛发展,公司或组织业务的不断折扩张,需求的快速变化以及
用户量的不断增加,传统的单块(Monolithic)软件架构面临高着越来越多的挑战,已逐渐
无法适应互联网时代对软件的要求。在这一背景下,微服务架构构模式(Microservice
Architecture Pattern)逐渐流行,它强调将单一业务功能开发成微服务的形式,每个微
服务运行在一个进程中;采用HTTP等通用协议和轻量级API实现微服务之间的协作与通
信。这些微服务可以使用不同的开发语言以及不同数据存储技术,能够通过自动化部署工
具独立发布,并保持最低限制的集中式管理。
请围绕"微服务系统开发应用"论题,依次从以下三个方面进行评论述。
1.概要叙述你参与管理和开发的、采用微服务架构的软件开发项目及在其中所担任的主要
工作。
2.在微服务架构被提出之前,传统的Web开发方式为单体式开发。对比介绍单体式系统与
微服务系统之间的区别。
3.结合你参与管理和开发的软件开发项目,说明该项目是如何采用微服务架构模式的,并
说明在采用微服务架构后,在软件开发过程中遇到的实际问题和0解决方案。

网 关收到前 端的请 ， 求不会直接调用后 端的业务服务，而是首先 会 从服务 注册中心根据当前请求 来获取对应
的服务配置，随后通过服务配置 再调用已注册的服务。当后 端微服务存在多个实例，时将采取负载均衡的方式 调用 。服务注册中心是整个 微服务架构体系的 核心部 分，由 Spring Cloud 的Eureka 组件 来实现，专门

提供 微服务的服务 注册和发现 功能 ， 涉及三种角色: 服务提供 者 、服务消费者和服务注册中心 。API 路由网

关、所有业务服务 ，以及服务监控平台组件 都注册到服务 注册中心。通过服务注册中心 两两互相注册、API

路 由网关向服务注册中心注册多个实例等 方式，来实现 后端 整体服务的 高可靠 性 。服务 监 控平台通过注册到
服务注册中 心，获取所有注册到服务注册中心 的后端业务服务，从而监控到所有后 端业务服务的运行状态信息，最后收集并展示整个 微服务 系统的运行 状态，更进一步保证 整个 后 端的服务质量。

三、业务服务
业务服务按系统业务 模块，相应划分为 题库服务 、评测机服务、实验作业服务、考试服务 、比赛服务、

抄 效判定服务、MQ 队列服务 等 。服务间协同 工作, 通过松 耦合的服务发现机制, 来动态调用对方的REST API

接口。其 中，对于压力较 大的服务，如评测机服务 、抄 袭判定服务 ， 等将部署为多实例集群 。以在线考试功能 为例，用户进入考试时，考试服务核验考生信息 通过 ， 后根据预先设置 的参数，调用题库服务，题库服务返回试题信息，由考试服务组合 为试卷，返回前 端显 示。用户交卷时，提交的程序代码到达考 试服务，考试服务拆分后分 发 给题库服务，题库服务将 程序 代码和测试用例 送入MQ 队列排队。然 后由负载 均衡 机制，依
次 将队列中排队的 待评测程序分 发 给空闲的评测机服务 编译、执行、判分 ， 评测机服务完成评测，后将结果返回 给 题库服务和考试服务,题库服 务 进行试题通过率统计,考试 服务 进行 成绩 统计 ， 以及向前 端显示成绩。
在 这期间服务请求 者 无需了解其他服务对数据 如何具体 处理和分析。

系统自** 年近 月 正式上线已运行 一年有余，在芝中投入使用，截至目前已有 ** 以上的学生用户 、评测了
“* 条 以上的程序代码，获得了“的 一致好评。在开发和试运行过 程，中主要遇到了两个问题。一 是跨域问题。

“系统前后端分离，前 端通过 Ajax 访问后端服务。由于浏览器同源策略的限制，导致 前端UI 无法正常访问

不同端口和IP 的后 端服务。我们利用Spring

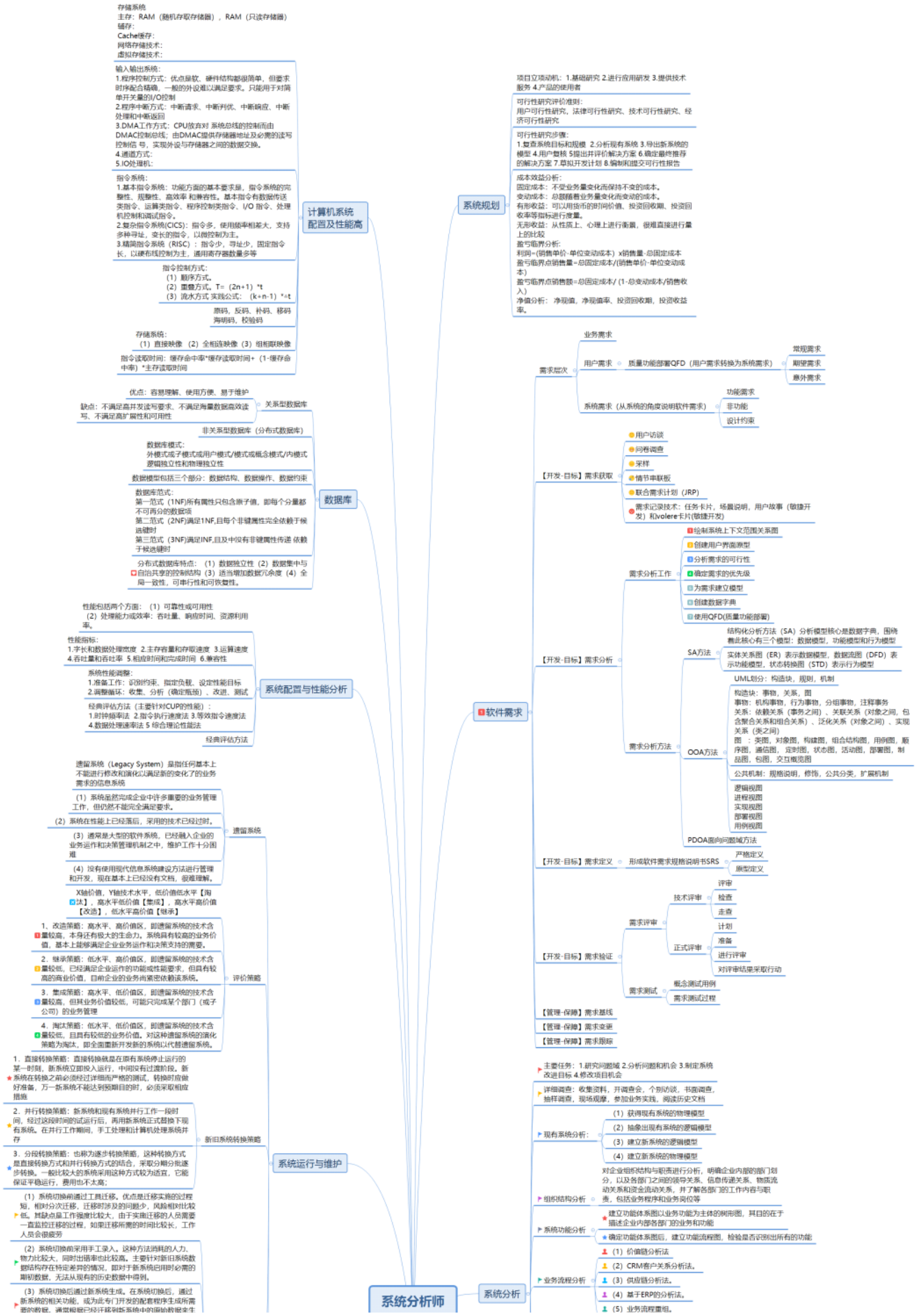
Boot 后端的Cors 跨域机制解决了该问题。二是评测机宕机问
题 。评 测机服务需要执行 用户提交的代码，但由于部分用户 短 时间内提交了大量 不安全代码，导致所有评测机服务全部宕机。我们引入心跳机制、快照回 滚机制，以及基于机器学习技术的预判断机制，使评测服务宕机时能够在10 秒内自动 重置恢复 运行，最终解了决该问题。实 践证明，“ 系统项目能 够顺利上线，并且稳定 运行，与系统采用了合适的架构设计密不可 分。经过这次微服 务架构应用的 方法和实施的效果 ， 后我也看

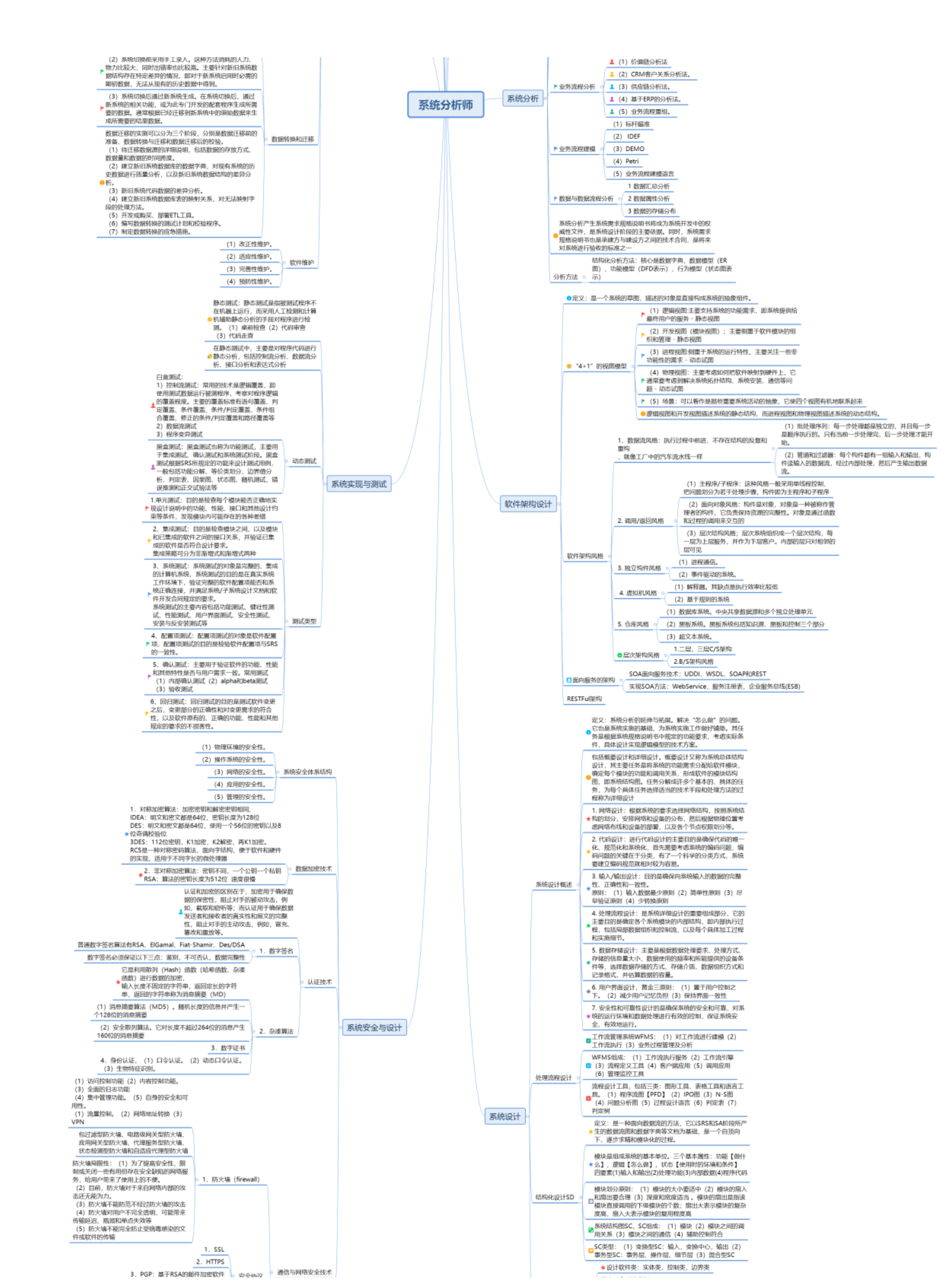
到了自己身上的不足之处, 在未来还会不断地更新知识, 完善本系统的架构设计, 使 整个系统能 够更加好用,

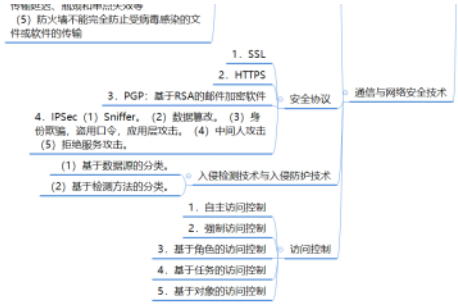
更 有效地服务于高校师生。

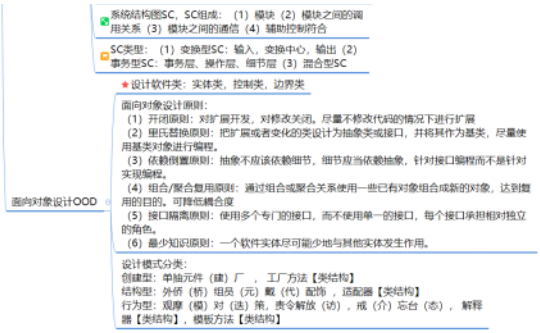
思维导图

2025年5月20日 17:07




系统分析师
系统分析
业务流程分析
（1）价值链分析法
（2）CRM客户关系分析法。
（3）供应链分析法。
（4）基于ERP的分析法。
（5）业务流程重组。
业务流程建模
（1）标杆瞄准
（2）IDEF
（3）DEMO
（4）Petri
（5）业务流程建模语言
数据与数据流程分析
1 数据汇总分析
2 数据属性分析
3 数据的存储分布
系统分析产生系统需求规格说明书将成为系统开发中的权威性文件，是系统设计阶段的主要依据。同时，系统需求规格说明书也是承建方与建设方之间的技术合同，是将来对系统进行验收的标准之一
分析方法
结构化分析方法：核心是数据字典，数据模型（ER图），功能模型（DFD表示），行为模型（状态图表示）
软件架构设计
定义：是一个系统的草图，描述的对象是直接构成系统的抽象组件。
“4+1”的视图模型
（1）逻辑视图：主要支持系统的功能需求，即系统提供给最终用户的服务 - 静态视图
（2）开发视图（模块视图）：主要侧重于软件模块的组织和管理 - 静态视图
（3）进程视图：侧重于系统的运行特性，主要关注一些非功能性的需求 - 动态试图
（4）物理视图：主要考虑如何把软件映射到硬件上，它通常要考虑到解决系统拓扑结构、系统安装、通信等问题 - 动态试图
（5）场景：可以看作是那些重要系统活动的抽象，它使四个视图有机地联系起来
逻辑视图和开发视图描述系统的静态结构，而进程视图和物理视图描述系统的动态结构。
软件架构风格
1、数据流风格：执行过程中前进，不存在结构的反复和重构，就像工厂中的汽车流水线一样
（1）批处理序列：每一步处理都是独立的，并且每一步是顺序执行的。只有当前一步处理完，后一步处理才能开始。
（2）管道和过滤器：每个构件都有一组输入和输出，构件读输入的数据流，经过内部处理，然后产生输出数据流。
2. 调用/返回风格
（1）主程序/子程序：这种风格一般采用单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序
（2）面向对象风格：构件是对象，对象是一种被称作管理者的构件，它负责保持资源的完整性。对象是通过函数和过程的调用来交互的
（3）层次结构风格：层次系统组织成一个层次结构，每一层为上层服务，并作为下层客户。内部的层只对相邻的层可见
3. 独立构件风格
（1）进程通信。
（2）事件驱动的系统。
4. 虚拟机风格
（1）解释器。其缺点是执行效率比较低
（2）基于规则的系统
5. 仓库风格
（1）数据库系统。中央共享数据源和多个独立处理单元
（2）黑板系统。黑板系统包括知识源、黑板和控制三个部分
（3）超文本系统。
层次架构风格
1.二层，三层C/S架构
2.B/S架构风格
面向服务的架构
SOA面向服务技术：UDDI、WSDL、SOAP和REST
实现SOA方法：WebService，服务注册表，企业服务总线(ESB)
RESTful架构
系统设计
系统设计概述
定义：系统分析的延伸与拓展。解决“怎么做”的问题。它也是系统实施的基础，为系统实施工作做好铺垫。其任务是根据系统规格说明书中规定的功能要求，考虑实际条件，具体设计实现逻辑模型的技术方案。
包括概要设计和详细设计。概要设计又称为系统总体结构设计，其主要任务是将系统的功能需求分配给软件模块，确定每个模块的功能和调用关系，形成软件的模块结构图，即系统结构图。任务分解成许多个基本的、具体的任务，为每个具体任务选择适当的技术手段和处理方法的过程称为详细设计
1. 网络设计：根据系统的要求选择网络结构，按照系统结构的划分，安排网络和设备的分布，然后根据物理位置考虑网络布线和设备的部署，以及各个节点权限划分等。
2. 代码设计：进行代码设计的主要目的是确保代码的唯一化、规范化和系统化。首先需要考虑系统的编码问题，编码问题的关键在于分类。有了一个科学的分类方式，系统要建立编码规范就相对较为容易。
3. 输入/输出设计：目的是确保向系统输入的数据的完整性、正确性和一致性。
原则：（1）输入数据最少原则（2）简单性原则（3）尽早验证原则（4）少转换原则
4. 处理流程设计：是系统详细设计的重要组成部分，它的主要目的是确定各个系统模块的内部结构，即内部执行过程，包括局部数据组织和控制流，以及每个具体加工过程和实施细节。
5. 数据存储设计：主要是根据数据处理要求、处理方式、存储的信息量大小、数据使用的频率和所能提供的设备条件等，选择数据存储的方式、存储介质、数据组织方式和记录格式，并估算数据的容量。
6. 用户界面设计，黄金三原则：（1）置于用户控制之下。（2）减少用户记忆负担（3）保持界面一致性
7. 安全性和可靠性设计的是确保系统的安全和可靠，对系统的运行环境和数据处理进行有效的控制，保证系统安全、有效地运行。
处理流程设计
工作流管理系统WFMS：（1）对工作流进行建模（2）工作流执行（3）业务过程管理及分析
WFMS组成：（1）工作流执行服务（2）工作流引擎（3）流程定义工具（4）客户端应用（5）调用应用（6）管理监控工具
流程设计工具，包括三类：图形工具、表格工具和语言工具。（1）程序流图【PFD】（2）IPO图（3）N-S图（4）问题分析图（5）过程设计语言（6）判定表（7）判定树
结构化设计SD
定义：是一种面向数据流的方法，它以SRS和SA阶段所产生的数据流图和数据字典等文档为基础，是一个自顶向下、逐步求精和模块化的过程。
模块是组成系统的基本单位。三个基本属性：功能【做什么】，逻辑【怎么做】，状态【使用时的环境和条件】
四要素(1)输入和输出(2)处理功能(3)内部数据(4)程序代码
模块划分原则：（1）模块的大小要适中（2）模块的扇入和扇出要合理（3）深度和宽度适当。模块的扇出是指该模块直接调用的下级模块的个数；扇出大表示模块的复杂度高，扇入大表示模块的复用程度高
系统结构图SC，SC组成：（1）模块（2）模块之间的调用关系（3）模块之间的通信（4）辅助控制符合
SC类型：（1）变换型SC：输入，变换中心，输出（2）事务型SC：事务层、操作层、细节层（3）混合型SC
设计软件类：实体类，控制类，边界类
系统实现与测试
数据转换和迁移
（2）系统切换前采用手工录入。这种方法消耗的人力、物力比较大，同时出错率也比较高。主要针对新旧系统数据结构存在特定差异的情况，即对于新系统启用时必需的期初数据，无法从现有的历史数据中得到。
（3）系统切换后通过新系统生成。在系统切换后，通过新系统的相关功能，或为此专门开发的配套程序生成所需要的数据。通常根据已经迁移到新系统中的原始数据来生成所需要的结果数据。
数据迁移的实施可以分为三个阶段，分别是数据迁移前的准备、数据转换与迁移和数据迁移后的校验。
（1）待迁移数据源的详细说明，包括数据的存放方式、数据量和数据的时间跨度。
（2）建立新旧系统数据库的数据字典，对现有系统的历史数据进行质量分析，以及新旧系统数据结构的差异分析。
（3）新旧系统代码数据的差异分析。
（4）建立新旧系统数据库表的映射关系，对无法映射字段的处理方法。
（5）开发或购买、部署ETL工具。
（6）编写数据转换的测试计划和校验程序。
（7）制定数据转换的应急措施。
软件维护
（1）改正性维护。
（2）适应性维护。
（3）完善性维护。
（4）预防性维护。
测试类型
静态测试：静态测试是指被测试程序不在机器上运行，而采用人工检测和计算机辅助静态分析的手段对程序进行检测。（1）桌前检查（2）代码审查（3）代码走查
在静态测试中，主要是对程序代码进行静态分析，包括控制流分析、数据流分析、接口分析和表达式分析
白盒测试：
1）控制流测试：常用的技术是逻辑覆盖，即使用测试数据运行被测程序，考察对程序逻辑的覆盖程度。主要的覆盖标准有语句覆盖、判定覆盖、条件覆盖、条件/判定覆盖、条件组合覆盖、修正的条件/判定覆盖和路径覆盖等
2）数据流测试
3）程序变异测试
动态测试
黑盒测试：黑盒测试也称为功能测试，主要用于集成测试、确认测试和系统测试阶段。黑盒测试根据SRS所规定的功能来设计测试用例，一般包括功能分解、等价类划分、边界值分析、判定表、因果图、状态图、随机测试、错误推测和正交试验法等
1.单元测试：目的是检查每个模块能否正确地实现设计说明中的功能、性能、接口和其他设计约束等条件，发现模块内可能存在的各种差错
2、集成测试：目的是检查模块之间，以及模块和已集成的软件之间的接口关系，并验证已集成的软件是否符合设计要求。
集成策略可分为非渐增式和渐增式两种
3、系统测试：系统测试的对象是完整的、集成的计算机系统。系统测试的目的是在真实系统工作环境下，验证完整的软件配置项能否和系统正确连接，并满足系统/子系统设计文档和软件开发合同规定的要求。
系统测试的主要内容包括功能测试、健壮性测试、性能测试、用户界面测试、安全性测试、安装与反安装测试等
4、配置项测试：配置项测试的对象是软件配置项，配置项测试的目的是检验软件配置项与SRS的一致性。
5、确认测试：主要用于验证软件的功能、性能和其他特性是否与用户需求一致。常用测试（1）内部确认测试（2）alpha和beta测试（3）验收测试
6、回归测试：回归测试的目的是测试软件变更之后，变更部分的正确性和对变更需求的符合性，以及软件原有的、正确的功能、性能和其他规定的要求的不损害性。
系统安全与设计
系统安全体系结构
（1）物理环境的安全性。
（2）操作系统的安全性。
（3）网络的安全性。
（4）应用的安全性。
（5）管理的安全性。
数据加密技术
1. 对称加密算法：加密密钥和解密密钥相同。
IDEA：明文和密文都是64位，密钥长度为128位
DES：明文和密文都是64位，使用一个56位的密钥以及8位奇偶校验位
3DES：112位密钥，K1加密，K2解密，再K1加密。
RC5是一种对称密码算法，面向字结构，便于软件和硬件的实现，适用于不同字长的微处理器
2. 非对称加密算法：密钥不同，一个公钥一个私钥
RSA：算法的密钥长度为512位 速度很慢
认证和加密的区别在于，加密用于确保数据的保密性，阻止对手的被动攻击，例如，截取和窃听等；而认证用于确保数据发送者和接收者的真实性和报文的完整性，阻止对手的主动攻击，例如，冒充、篡改和重放等。
认证技术
1、数字签名
普通数字签名算法有RSA、ElGamal、Fiat-Shamir、Des/DSA
数字签名必须保证以下三点：鉴别，不可否认，数据完整性
2、杂凑算法
它是利用散列（Hash）函数（哈希函数、杂凑函数）进行数据的加密，
输入长度不固定的字符串，返回定长的字符串，返回的字符串称为消息摘要（MD）
（1）消息摘要算法（MD5）。随机长度的信息并产生一个128位的消息摘要
（2）安全散列算法。它对长度不超过264位的消息产生160位的消息摘要
3、数字证书
4、身份认证。（1）口令认证。（2）动态口令认证。（3）生物特征识别。
通信与网络安全技术
（1）访问控制功能（2）内容控制功能。
（3）全面的日志功能
（4）集中管理功能。（5）自身的安全和可用性。
（1）流量控制。（2）网络地址转换（3）VPN
1、防火墙（firewall）
包过滤型防火墙、电路级网关型防火墙、应用网关型防火墙、代理服务型防火墙、状态检测型防火墙和自适应代理型防火墙
防火墙局限性：（1）为了提高安全性，限制或关闭一些有用但存在安全缺陷的网络服务，给用户带来了使用上的不便。
（2）目前，防火墙对于来自网络内部的攻击还无能为力。
（3）防火墙不能防范不经过防火墙的攻击
（4）防火墙对用户不完全透明，可能带来传输延迟、瓶颈和单点失效等
（5）防火墙不能完全防止受病毒感染的文件或软件的传输
安全协议
1、SSL
2、HTTPS
3、PGP：基于RSA的邮件加密软件


传输层后，能够起到防止关键
（5）防火墙不能完全防止受病毒感染的文件或软件的传输
通信与网络安全技术
安全协议
1．SSL
2．HTTPS
3．PGP：基于RSA的邮件加密软件
4．IPSec（1）Sniffer，（2）数据篡改，（3）身份欺骗，盗用口令，应用层攻击。（4）中间人攻击（5）拒绝服务攻击。
入侵检测技术与入侵防护技术
（1）基于数据源的分类。
（2）基于检测方法的分类。
访问控制
1．自主访问控制
2．强制访问控制
3．基于角色的访问控制
4．基于任务的访问控制
5．基于对象的访问控制


面向对象设计OOD
系统结构图SC，SC组成：（1）模块（2）模块之间的调用关系（3）模块之间的通信（4）辅助控制符合
SC类型：（1）变换型SC：输入，变换中心，输出（2）事务型SC：事务层，操作层，细节层（3）混合型SC
设计软件类：实体类，控制类，边界类
面向对象设计原则：
（1）开闭原则：对扩展开发，对修改关闭。尽量不修改代码的情况下进行扩展
（2）里氏替换原则：把扩展或者变化的类设计为抽象类或接口，并将其作为基类，尽量使用基类对象进行编程。
（3）依赖倒置原则：抽象不应该依赖细节，细节应当依赖抽象，针对接口编程而不是针对实现编程。
（4）组合/聚合复用原则：通过组合或聚合关系使用一些已有对象组合成新的对象，达到复用的目的。可降低耦合度
（5）接口隔离原则：使用多个专门的接口，而不使用单一的接口，每个接口承担相对独立的角色。
（6）最少知识原则：一个软件实体尽可能少地与其他实体发生作用。
设计模式分类：
创建型：单抽元件（建）厂 ， 工厂方法【类结构】
结构型：外侨（桥）组员（元）戴（代）配饰 ，适配器【类结构】
行为型：观摩（模）对（迭）策，责令解放（访），戒（介）忘台（态）， 解释器【类结构】，模板方法【类结构】

Web系统测试

2025年5月21日 15:28

论Web系统的测试技术及其应用

随着网络技术的广泛应用,许多传统的信息系统已经逐渐被移植到互联网上,Web系统已经对日常的工作和生活产生了深远的影响。为了保证Web系统的正确性,在系统开发阶段就要对其进行全面的测试、确认和验收,而且由于Web系统具有与传统信息系统截然不同的特点,需要采用针对Web系统特点的测试技术与方法。

问题内容:

请围绕"Web系统的测试技术及其应用"论题,依次从以下三个方面进行论述。

1.概要叙述你参与实施的Web系统开发项目以及你所担任的主要工作。

2.阐述主要针对哪几个方面进行Web系统测试,并对每个方面面的具体测试内容进行详细论述。

3.阐述在进行Web系统的测试时遇到了哪些问题,如何解决。

智慧办公平台论文框架

2025年10月14日 11:36

摘要（300字）：2022年8月，我公司受某运营商业务支撑中心委托，开展智慧办公平台系统的建设工作，我作为系统分析师负责系统的分析设计以及项目的实施工作。该系统主要功能包括：考勤考核、门禁管理、物业服务、会议预定、音频转录、AI扫描、智慧打印、餐厅服务等模块，将智慧办公理念贯穿于一体化全流程服务中，有效提高办公效率与管理水准，成为全省首个AI智慧办公的园区示范。本人笔者结合实际工作经验就该项目的（需求分析）展开讨论和分析，讨论了……方法，阐述了……应用。实践证明，使用……技术可以提高软件开发的成功率。该项目总投入878万元，历时20个月，于2024年交付运行至今，广受集团领导和园区内员工的一致好评，为后续打造同类的智慧办公园区提供了可参考的标杆范式。

项目背景：随着办公点二期三期园区的建成、园区承载员工人数日益增加，加之当前系统分散，管理不便、信息不互通等问题日益显现，成为掣肘办公效率的重要影响因素。原有多套系统分别建设开发，例如员工一次申请审批需求需要分别在多套系统中提交、人工跟踪提醒每一位审批人进行登录系统进行审批；新入场的员工需要在多套系统中建立数据、不同园区的访客入场审批需要在不同的系统中进行操作。门禁管理、物业管理、会议管理、文档流转等高效办公的需求日益迫切，业务支撑中心领导决定建设一套综合的智慧办公系统平台，依托新建的智慧园区，赋能智慧办公能力，将所有分散的信息流集成在一个系统中，让所有的员工在一套智慧办公系统平台中，就可以享受整个园区所有的智慧办公能力。该系统提供的智慧能力遍及办公园区方方面面的办公场景，打破各自为政的局面，实现全园区数据互联互通、信息充分共享利用、提高数据的使用效率和利用效率。

项目整体介绍：我所在的公司提交的建设方案获得该运营商业务支撑中心领导的认可，成功中标该项目，并于2022年8月正式启动智慧办公系统平台的建设项目。我作为系统分析师，负责系统分析和设计工作，将业务需求与技术架构的深度融合，确保建设的系统平台与项目需求相吻合。该项目总投入878万元，自2022年8月到2024年4月，历时20个月，系统主要部署在某园区数据中心云资源池中，便于弹性伸缩，并在各园区数据中心部署负载均衡和缓存冗余节点，保证智慧办公系统平台的可用性，保障各园区办公活动的有序进行。项目建设主要包含几大模块：一是人员管理模块，接入账号统一管理平台，对园区人员进行统一的管理，同时设置访客子模块，对访客身份和权限进行有效管理。二是物业服务，对用餐、打印、维修等物业服务能力进行智慧能力改造升级，罗列清晰明了的成本费用，自动提交报销、划账等审批流程；三是会议能力，提供会议室预定，会议提醒通知，会议音视频投放，远程会议间，AI文档扫描，会议实时字幕、文字转录的强大的功能，利用AI赋能会议高效进行。

笔者所在的公司虽然在其他的办公平台项目取得成功，但该项目涉及范围较广，使用了AI进行赋能，具有严格的权限划分和审批要求，在建设开发测试的过程中，团队成员进行每一步敏感操作都要进行严格的金库审批才能进行，这无疑增加了该系统建设的难度。于是笔者决定使用（引出正文）

结尾：经过20个月的努力开发建设，该智慧办公系统平台于2023年4月投入使用，支撑三个办公点，七个办公区，两万余人的日常办公需求。系统经过了全面的功能测试、安全测试、性能测试，运行至今客户反馈良好。由于系统规模大，涉及范围广，性能要求高，项目建设周期长等因素，项目建设的过程中困难重重。笔者和项目团队成员非常重视项目的（），最终保证了项目顺利交维。

当然，在本项目的建设过程中，也存在一些不足之处，在项目完成后的总结研讨会上，大家也指出了本次建设项目中存在的一些不足：例如存在一些个人敏感信息未脱敏，一些开放的页面存在未授权访问等问题，不符合合规的要求，这些都在项目后期的安全测试中得以纠正，并没有对项目产生实质性的影响。该项目的顺利完成，也给笔者提供了一次成功经验，在未来的工作中，我也将继续不断学习，自我完善，不断提升自己的专业能力，以期在未来更好的完成系统分析和设计工作。

默打

2025年10月15日 15:37

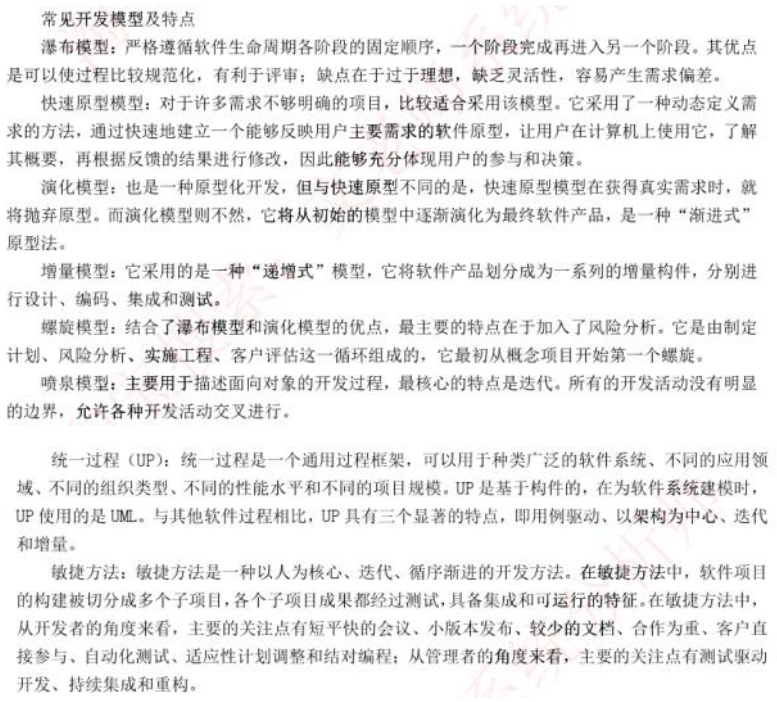
押题：需求分析

2025年10月15日 17:05

常见开发模型及其特点

2025年10月21日 15:21

瀑布模型:严格遵循软件生命周期各阶段的固定顺序,一个阶段完成再进入另一个阶段。其优点是可以使过程比较规范化,有利于评审;缺点在于过于理想,缺乏灵活性,容易产生需求偏差,难以适应需求的变化。
快速原型模型:对于许多需求不够明确的项目,比较适合采用该模型。它采用了一种动态定义需求的方法,通过快速地建立一个能够反映用户主要需求的软件原型,让用户在计算机上使用它,了解其概要,再根据反馈的结果进行修改,因此能够充分体现用户的参与和决策。
演化模型:也是一种原型化开发,但与快速原型不同的是,快速原型模型在获得真实需求时,就将抛弃原型。而演化模型则不然,它将从初始的模型中逐渐演化为最终软件产品,是一种"渐进式"原型法，适用于用户需求不明确的项目。
增量模型:它采用的是一种"递增式"模型,它将软件产品划分成为一系列的增量构件,分别进行设计、编码、集成和测试。
螺旋模型:结合了瀑布模型和演化模型的优点,最主要的待点在于加入了风险分析。它是由制定计划、风险分析、实施工程、客户评估这一循环组成的,它最初从概念项目开始第一个螺旋，适合规模庞大复杂且风险较高的项目。
喷泉模型:主要用于描述面向对象的开发过程,最核心的特点是迭代。所有的开发活动没有明显的边界,允许各种开发活动交叉进行。
统一过程(UP):统一过程是一个通用过程框架,可以用于种类广泛的软件系统、不同的应用领域、不同的组织类型、不同的性能水平和不同的项目规模。UP是基于构件的,在为软件系统建模时,UP使用的是UML。与其他软件过程相比,UP具有三个显著的特点,即用例驱动、以架构（体系结构）为中心（4+1 用例视图）、迭代和增量。
分为四个阶段：初始 细化 构建 移交
敏捷方法:敏捷方法是一种以人为核心、迭代、循序渐进的开发方法，适合于小规模的项目开发。在敏捷方法中,软件项目的构建被切分成多个子项目,各个子项目成果都经过测试,具备集成和可运行的特征。在敏捷方法中,从开发者的角度来看,主要的关注点有短平快的会议、小版本发布、较少的文档、合作为重、客户直接参与、自动化测试、适应性计划调整和结对编程;从管理者的角度来看,主要的关注点有测试驱动开发、持续集成和重构。



常见开发模型及特点

瀑布模型：严格遵循软件生命周期各阶段的固定顺序，一个阶段完成再进入另一个阶段。其优点是可以使过程比较规范化，有利于评审；缺点在于过于理想，缺乏灵活性，容易产生需求偏差。

快速原型模型：对于许多需求不够明确的项目，比较适合采用该模型。它采用了一种动态定义需求的方法，通过快速地建立一个能够反映用户主要需求的软件原型，让用户在计算机上使用它，了解其概要，再根据反馈的结果进行修改，因此能够充分体现用户的参与和决策。

演化模型：也是一种原型化开发，但与快速原型不同的是，快速原型模型在获得真实需求时，就将抛弃原型。而演化模型则不然，它将从初始的模型中逐渐演化为最终软件产品，是一种“渐进式”原型法。

增量模型：它采用的是一种“递增式”模型，它将软件产品划分成为一系列的增量构件，分别进行设计、编码、集成和测试。

螺旋模型：结合了瀑布模型和演化模型的优点，最主要的特点在于加入了风险分析。它是由制定计划、风险分析、实施工程、客户评估这一循环组成的，它最初从概念项目开始第一个螺旋。

喷泉模型：主要用于描述面向对象的开发过程，最核心的特点是迭代。所有的开发活动没有明显的边界，允许各种开发活动交叉进行。

统一过程（UP）：统一过程是一个通用过程框架，可以用于种类广泛的软件系统、不同的应用领域、不同的组织类型、不同的性能水平和不同的项目规模。UP 是基于构件的，在为软件系统建模时，UP 使用的是 UML。与其他软件过程相比，UP 具有三个显著的特点，即用例驱动、以架构为中心、迭代和增量。

敏捷方法：敏捷方法是一种以人为核心、迭代、循序渐进的开发方法。在敏捷方法中，软件项目的构建被切分成多个子项目，各个子项目成果都经过测试，具备集成和可运行的特征。在敏捷方法中，从开发者的角度来看，主要的关注点有短平快的会议、小版本发布、较少的文档、合作为重、客户直接参与、自动化测试、适应性计划调整和结对编程；从管理者的角度来看，主要的关注点有测试驱动开发、持续集成和重构。